VISIONGUARD

AI-generated vs real image classification

MLOps Assignment 2
Data, training, tracking, serving, containers, CI, and cloud integration

Submission detail	Value
Name	Hayyan Haider
Roll number	23L-0727
Project	23L-0727_A2_Ops
Date	7 November 2026 (Asia/Karachi)
Graded source commit	214e6b7393212f03f1cbe61fe40c511cffd81d00
GitHub	https://github.com/HayyanHaider/23L-0727_A2_Ops
DagsHub	https://dagshub.com/HayyanHaider/23L-0727_A2_Ops
Colab notebook	https://colab.research.google.com/drive/1DUeUYKEAaNH7_sl6O8K48r6MESux_Htl
Scope	All eight required tasks. Optional Grad-CAM bonus not attempted.

AI assistance disclosure. OpenAI Codex helped prepare and repair the audit and training code, operated the Colab experiment, implemented and tested tracking and serving infrastructure, inspected evidence, and drafted this report. The reported measurements come from the actual assignment dataset and saved experiment outputs. The student should review the code, results, and interpretations before submission.

The experiment uses folder labels (AI=1, real=0). These labels are dataset annotations; a model score is not proof of image authenticity. All test figures reuse one inference pass per final model.

Task 1 - Data Understanding and Versioning

1.1 - Environment and GPU

The local project uses an isolated Python virtual environment, separating its dependencies from system Python. The graded model experiment ran in a separate Google Colab runtime with an NVIDIA T4 GPU. TensorFlow GPU detection and the runtime/package record are shown below; exact installed package versions are supplied in requirements-lock.txt.

Setting	Observed value
Python	3.13.16 (main, Oct 1 2026, 15:40:24) [GCC 13.3.0]
TensorFlow / Keras	2.21.0 / 3.13.2
GPU devices	/physical_device:GPU:0
CUDA / cuDNN	12.5.1 / 9
Random seed / image size / batch	42 / 224 x 224 RGB / 32

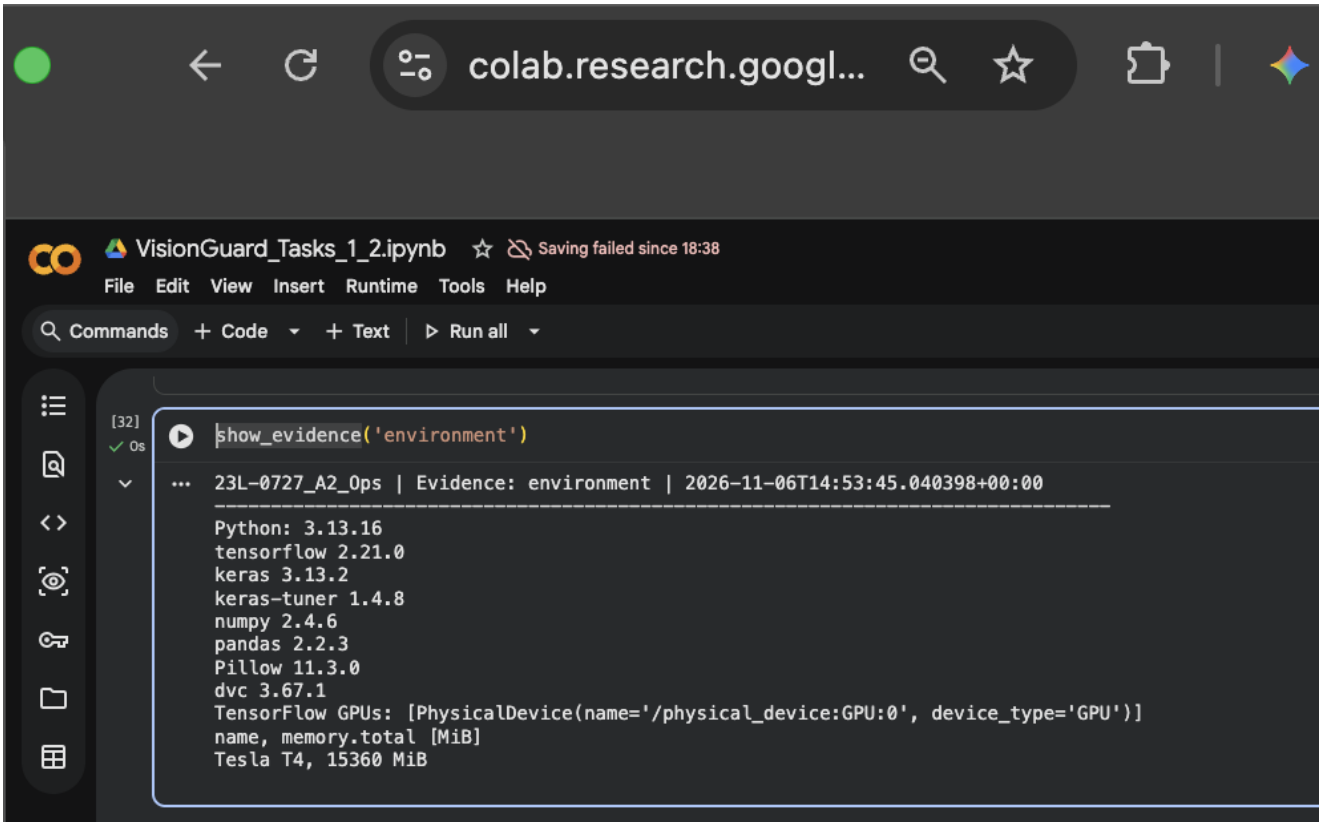


Figure 1. Actual Colab package versions and GPU evidence, with project context.

Commands and choices. python -m venv creates isolation; pip installs the declared dependencies. TensorFlow list_physical_devices checks GPU visibility; pip freeze records exact versions. Keeping Colab's supplied TensorFlow/CUDA combination avoids replacing its working GPU stack.

Observed issue. Google Drive was full, preventing notebook saves. Training checkpoints and outputs were therefore written under /content/23L-0727_evidence and downloaded to the local submission bundle. Runtime-local files alone are not persistent backups.

1.2 - Original dataset audit

The script recursively walks the original Kaggle files. For each candidate image it records class, file size, width, height, colour mode, format, byte SHA-256, decoded RGB pixel SHA-256, and any decoding error. Pillow verify plus a full decode checks readability. The nearest recognized parent folder assigns the label: AiArtData is AI and RealArt is real.

Class	Files	Readable	Width min/median/max	Height min/median/max
AI	538	538	225 / 1028.5 / 6000	135 / 800 / 4001
Real	435	435	183 / 940 / 8495	158 / 720 / 7091

Class	File bytes: minimum	Median	Maximum
AI	11,738	199,802.5	5,626,183
Real	11,311	144,982	10,195,352

There are 973 readable images and no unreadable candidates. One duplicate group contributes one extra file; decoded-pixel deduplication also removes one extra image. No cross-label pixel conflicts were found. AI images account for 55.3% of the original collection, so accuracy should be considered alongside precision, recall, F1, and ROC-AUC.

Split	Real	AI	Total
Train	304	376	680
Validation	65	81	146
Test	66	80	146

Deduplication happens before a seeded, stratified 70/15/15 split. Split lists retain original paths and hashes, making membership auditable. The retained pixel-hash sets are disjoint across train/validation/test. This only detects identical images: near duplicates and shared web sources may still leak visual information.

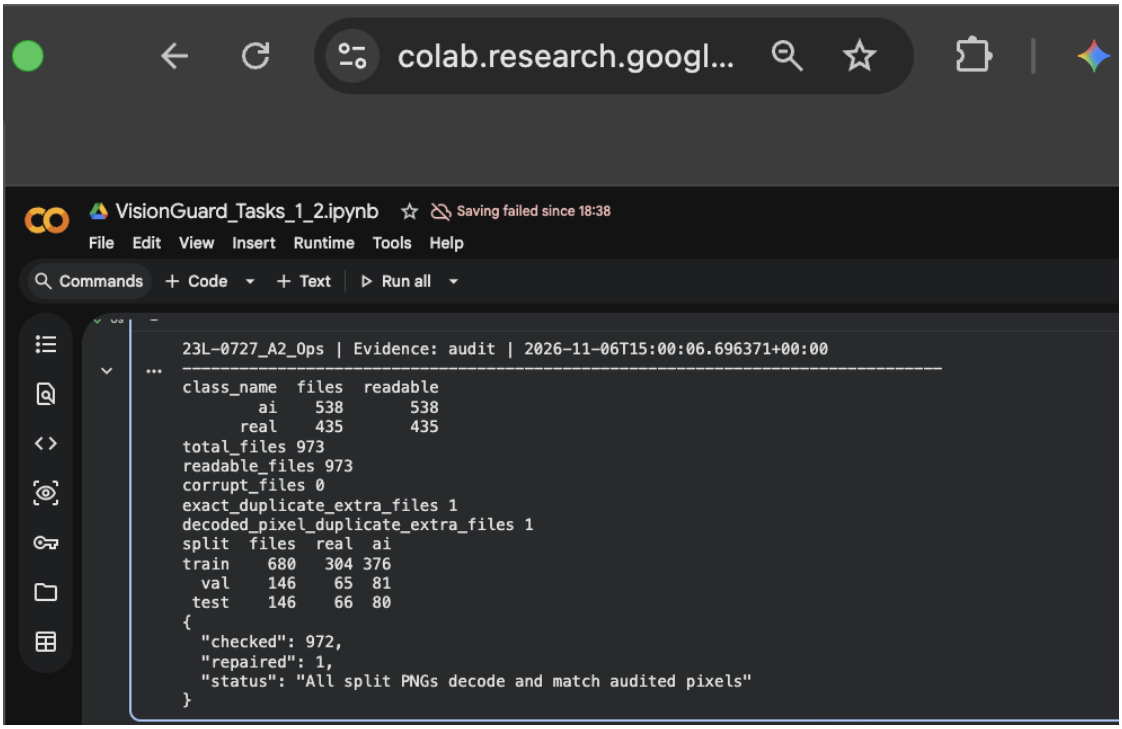


Figure 2. Executed audit and split evidence for the assignment dataset.

1.3 - Visual EDA: sample grid and widths

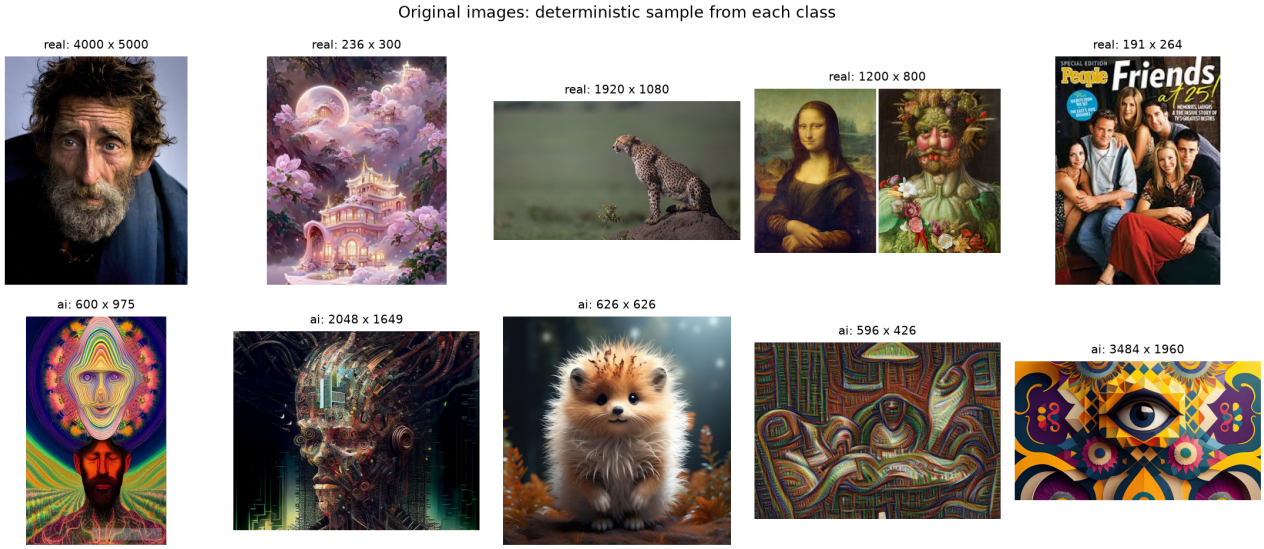


Figure 3. Deterministic original-image samples from each class (seed 42).

The real-labeled sample includes a portrait, a cheetah photograph, paintings, a magazine cover, and fantasy artwork; the AI sample includes stylized faces, abstract patterns, and a fluffy animal. This visible overlap means the problem is not simply photograph versus illustration, and the fantasy artwork in the real folder raises a label-provenance concern without proving mislabeling. Folder labels are retained as instructed; a small sample cannot establish authenticity or representativeness, and a separate label review would be needed.

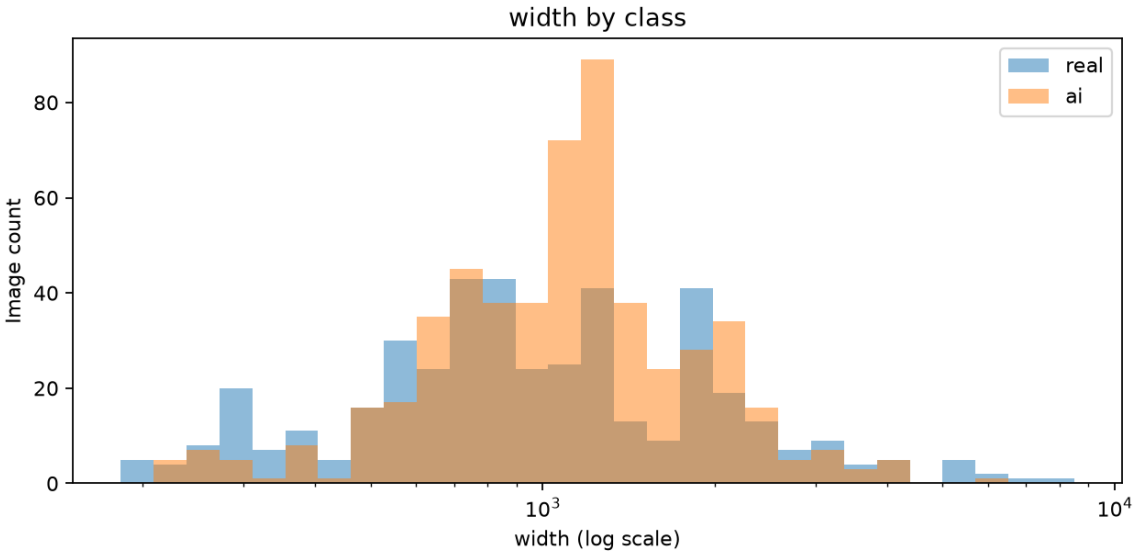


Figure 4. Original image width distribution by class; horizontal axis is logarithmic.

Median width is 1028.5 pixels for AI images and 940 pixels for real images, with substantial overlap. Real images have the more extreme maximum (8495 versus 6000 pixels). Resolution is therefore a possible source shortcut and compute cost; resizing makes tensor shapes consistent but does not establish that the detector has learned generation artifacts.

1.3 - Visual EDA: heights and file sizes

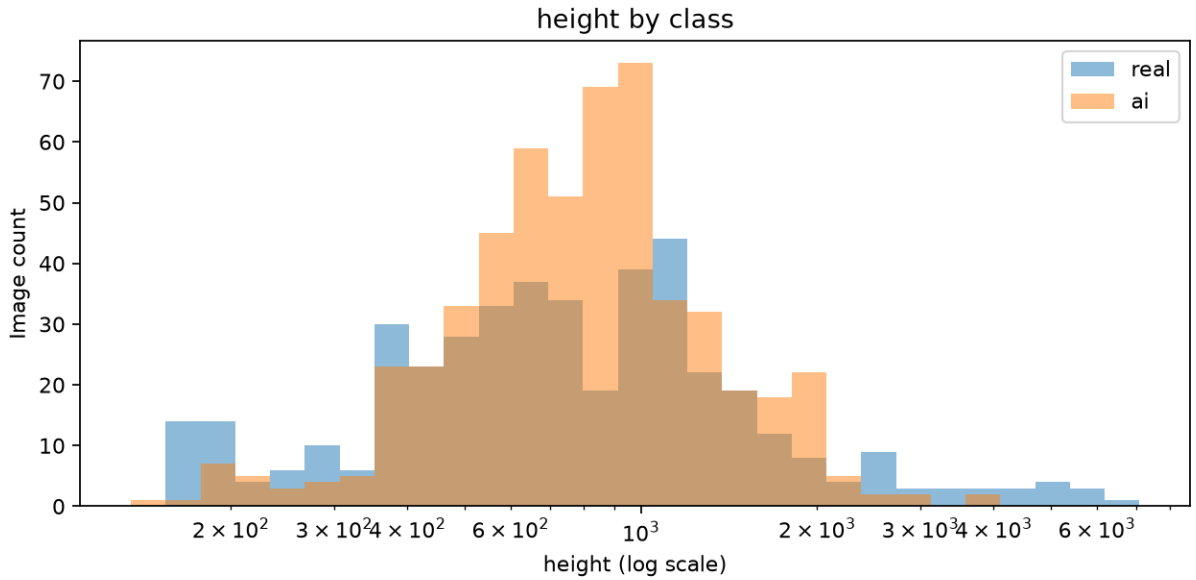


Figure 5. Original image heights by class on a logarithmic axis.

Median height is 800 pixels for AI images and 720 for real images. Both distributions span small and large images, and real images reach 7091 pixels compared with 4001 for AI. A fixed 224-pixel input reduces memory use, while potentially discarding fine forensic details.

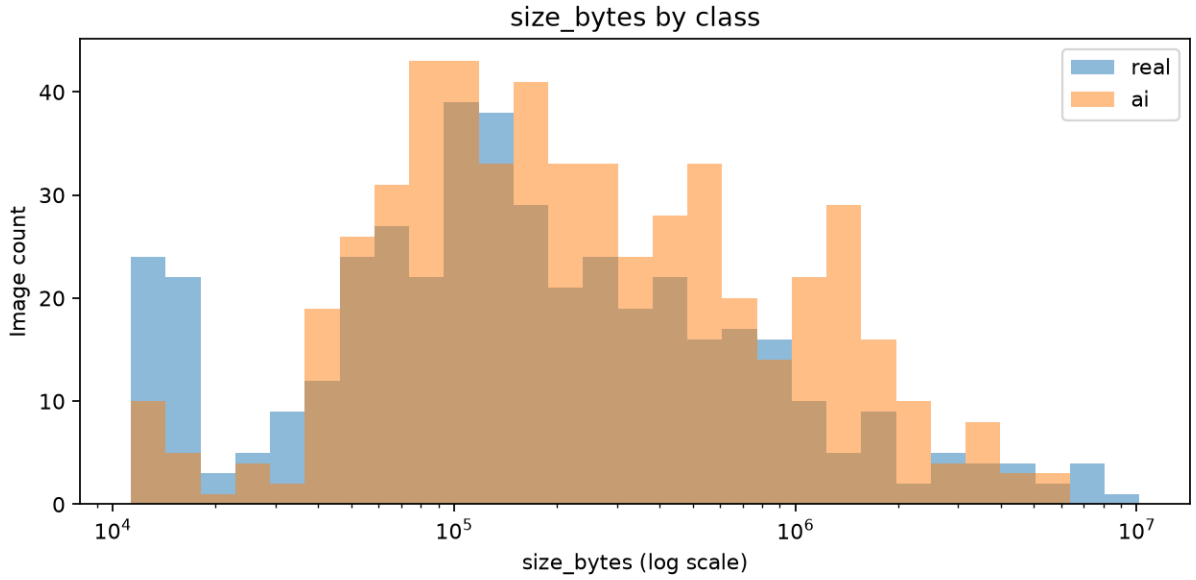


Figure 6. Original file-size distributions by class on a logarithmic axis.

The AI median file size is about 199.8 kB, compared with 145.0 kB for real images, although the largest real file is about 10.2 MB. These are original compressed file sizes, not processed PNG sizes or tensor memory. Compression format and source quality can correlate with class, so file size alone would be an unreliable authenticity signal.

1.4 - Git and DVC: toy round trip

A new empty directory was initialized with Git and DVC. tiny.csv initially contained two labeled rows; dvc add created its pointer, and Git committed the metadata. A third row was added and committed as a second version. Both data versions were pushed to a configured local-folder DVC remote, which the assignment permits.

Toy evidence	Observed value
Version 1 commit	1adeb67128fbb12396cfcd86f655fd17247261d6
Version 2 commit	ce8440c5e4f0da7f6365ea9250655d4185697450
Restored version	1adeb67128fbb12396cfcd86f655fd17247261d6
Recovery assertion	True

git checkout selected the first commit and dvc checkout restored its two-row content. The toy CSV and its local DVC cache were then removed. dvc pull recovered the older bytes from the remote, and the script asserted exact content equality. Deleting the cache makes this stronger than recovering from a local cached copy.

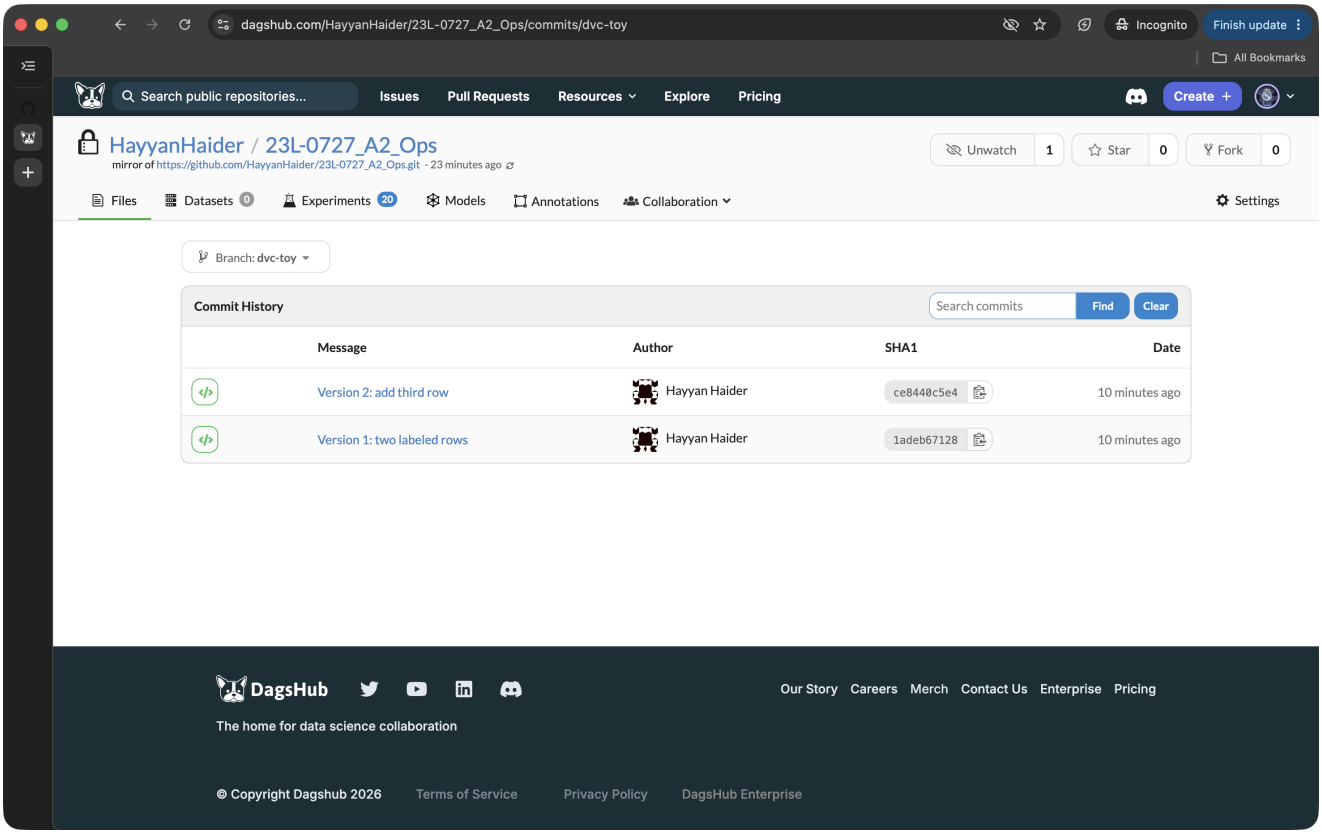


Figure 7. Both toy Git commits published on the DagsHub dvc-toy branch. Recovery is verified in the accompanying transcript and result.json.

git init starts metadata history; dvc init creates DVC configuration; dvc add hashes data and writes pointers; dvc remote add chooses object storage; dvc push uploads objects; dvc pull retrieves the objects named by the checked-out pointer. The two toy commits are also published to the DagsHub project as described in the accompanying repository evidence.

1.4 - Apply DVC to the real dataset

The original dataset, normalized RGB copies, and split manifests were tracked separately with DVC. dvc push uploaded their objects to the persistent local remote under versioned_storage/dvc_remote. Git stores the small pointers and configuration; a bare Git backup preserves metadata independently of the working tree.

Actual raw pointer:
outs:
- md5: b2cec67839f742df4653f66d4bcbbafb.dir
size: 503783378
nfiles: 975
hash: md5
path: raw

The pointer contains the content checksum, byte size, file count, hash algorithm, and path. For a directory, its .dir object identifies the file manifest. The audit uses SHA-256 for source identity; DVC's content checksum serves its own storage protocol. Large images remain outside Git because binary versions inflate history and do not produce useful text diffs.

The raw DVC pointer counts 975 files, while the image audit counts 973 candidates: DVC also includes two hidden .DS_Store files ignored by the image audit. This difference does not indicate two unreadable images.

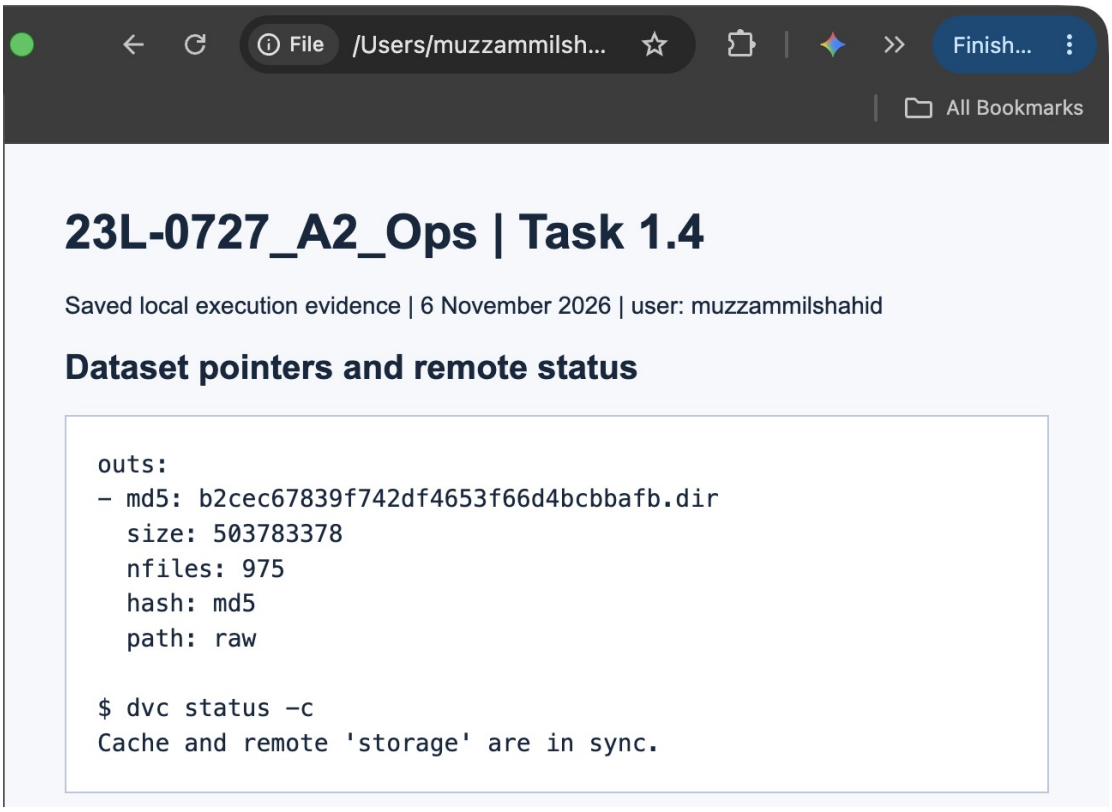


Figure 8. Browser view of the actual saved raw pointer and freshly executed DVC remote status. The log file is included in the evidence bundle.

1.4 - Published project and version history

Screenshots are genuine browser captures. New DagsHub evidence shows HayyanHaider; the displayed repository records and results are unchanged. macOS window captures exclude the software cursor overlay. The GitHub project is mirrored to DagsHub under the required roll-number project name. The dvc-toy branch preserves both CSV versions, while main contains the audited dataset pointers and model development source. Figure 9 shows the published project; large data objects remain in the permitted local DVC remote.

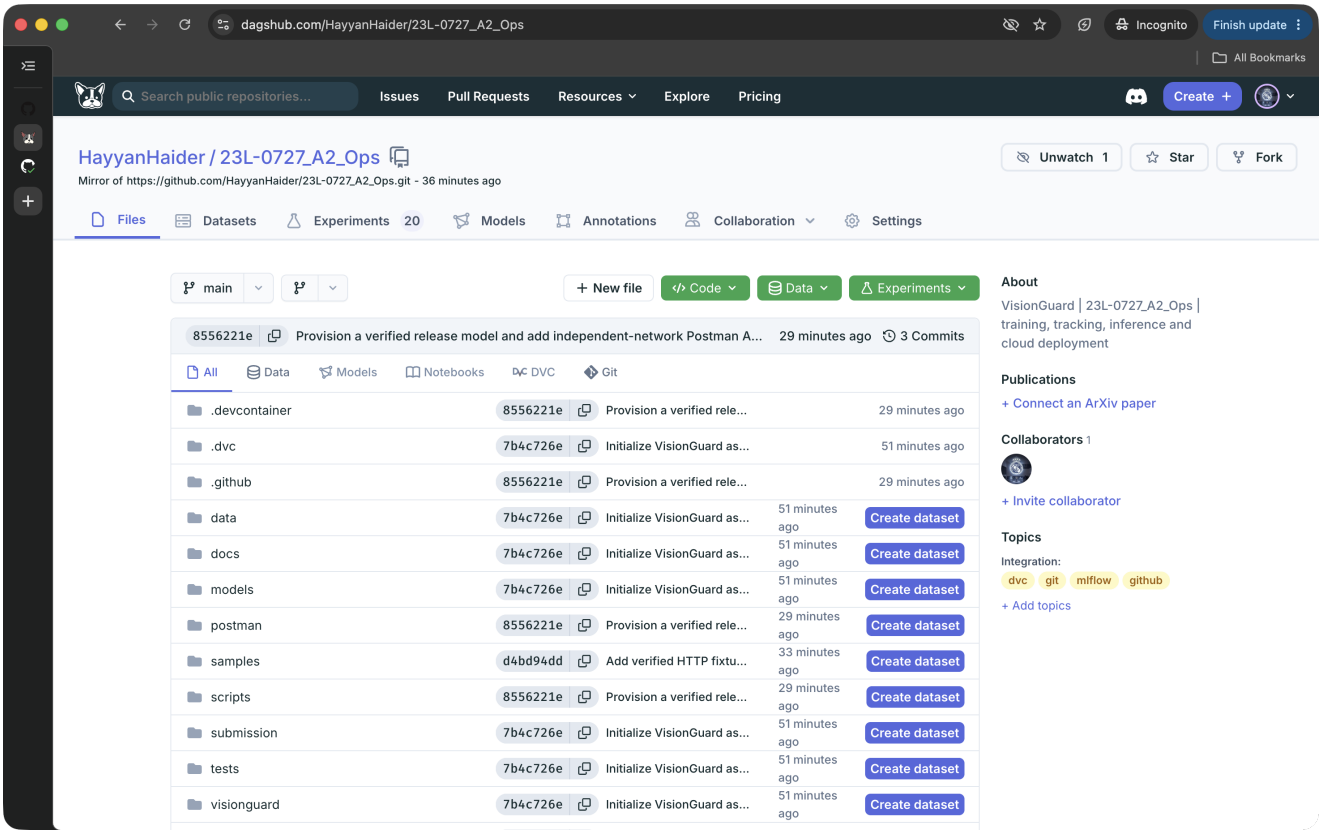


Figure 9. DagsHub repository excerpt: branch, source commit, and tracked files; account header cropped.

The local remote satisfies the permitted storage option. Reproducing on another computer requires access to that remote or an exported copy and an updated remote path; a Git clone alone cannot reconstruct excluded image bytes.

Task 2 - Model Development with TensorFlow/Keras

2.1 - Input pipeline and augmentation

Saved CSV split lists supply file paths and float labels. tf.data reads each RGB PNG, decodes three channels, resizes with bilinear interpolation and antialiasing, and pads to 224 x 224 without stretching. This matches MobileNetV2's standard input and limits GPU memory, at the cost of losing small artifacts. Padding can itself become a shortcut if aspect ratios correlate with labels.

The deterministic resized tensors are cached in memory before training shuffle. Training reshuffles each epoch with seed 42; batches contain up to 32 examples, and one batch is prefetched. Decode parallelism is capped at two to limit memory. Cache avoids repeatedly decoding large PNGs, and does not cache augmentation. Validation/test preserve manifest order and use no augmentation.

Training-only augmentation lives inside each model: horizontal flips, rotation up to 0.025 turns, zoom factor 0.08, and contrast factor 0.08. Reflect padding reduces blank fill during geometric augmentation. Keras disables these random transforms during evaluation; saved preprocessing therefore travels with the model.

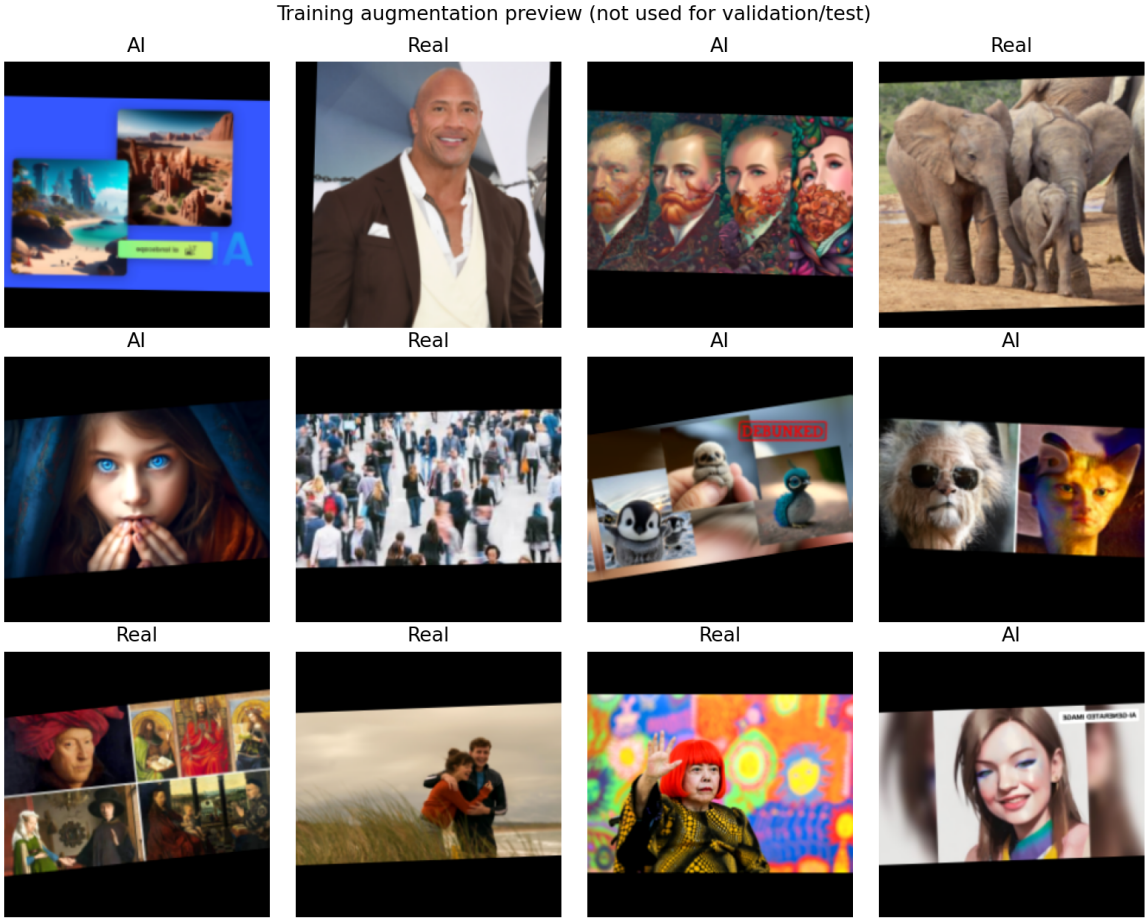


Figure 10. A genuine augmented training batch. Labels come from the manifest.

Observed repair. One processed PNG failed TensorFlow decoding. Its source byte hash and decoded pixel hash were checked against the original audit, then its RGB pixels were re-encoded with TensorFlow. All 972 split images passed decoding and pixel-hash verification; no examples or labels were dropped.

2.2 - Scratch CNN architecture

The scratch network has three convolution blocks (32, 64, 128 filters). Each 3 x 3 convolution shares local feature weights; BatchNorm stabilizes activations, ReLU adds nonlinearity, and max pooling reduces spatial size and computation. Global average pooling avoids a very large flattened classifier. A 64-unit ReLU layer combines channels; dropout 0.4 regularizes, and one sigmoid produces an AI score.

Layer	Output shape	Parameters
rgb_pixels_0_255	(None, 224, 224, 3)	0
train_only_augmentation	(None, 224, 224, 3)	0
scale_0_1	(None, 224, 224, 3)	0
conv_1	(None, 224, 224, 32)	864
bn_1	(None, 224, 224, 32)	128
relu_1	(None, 224, 224, 32)	0
pool_1	(None, 112, 112, 32)	0
conv_2	(None, 112, 112, 64)	18,432
bn_2	(None, 112, 112, 64)	256
relu_2	(None, 112, 112, 64)	0
pool_2	(None, 56, 56, 64)	0
conv_3	(None, 56, 56, 128)	73,728
bn_3	(None, 56, 56, 128)	512
relu_3	(None, 56, 56, 128)	0
pool_3	(None, 28, 28, 128)	0
global_average_pooling2d	(None, 128)	0
dense	(None, 64)	8,256
dropout	(None, 64)	0
p_ai	(None, 1)	65

The loss is binary cross entropy because the target is binary and the output is a sigmoid. Adam starts at 0.001, providing adaptive updates. Training monitors binary accuracy, AUC, precision, and recall; the latter classification metrics use 0.5 during fit. Final metrics instead use thresholds chosen solely from validation. Sigmoid scores are not guaranteed calibrated authenticity probabilities.

2.2 - Model summary evidence

Figure 11 shows the actual layer evidence exported from the trained scratch model. Shapes follow the data flow in the preceding table: three pooling operations reduce 224 x 224 to 28 x 28, and global average pooling reduces each feature map to one channel value. The screenshot focuses on layers with parameters. Convolutional layers dominate this small network; augmentation and pooling have no learned parameters.

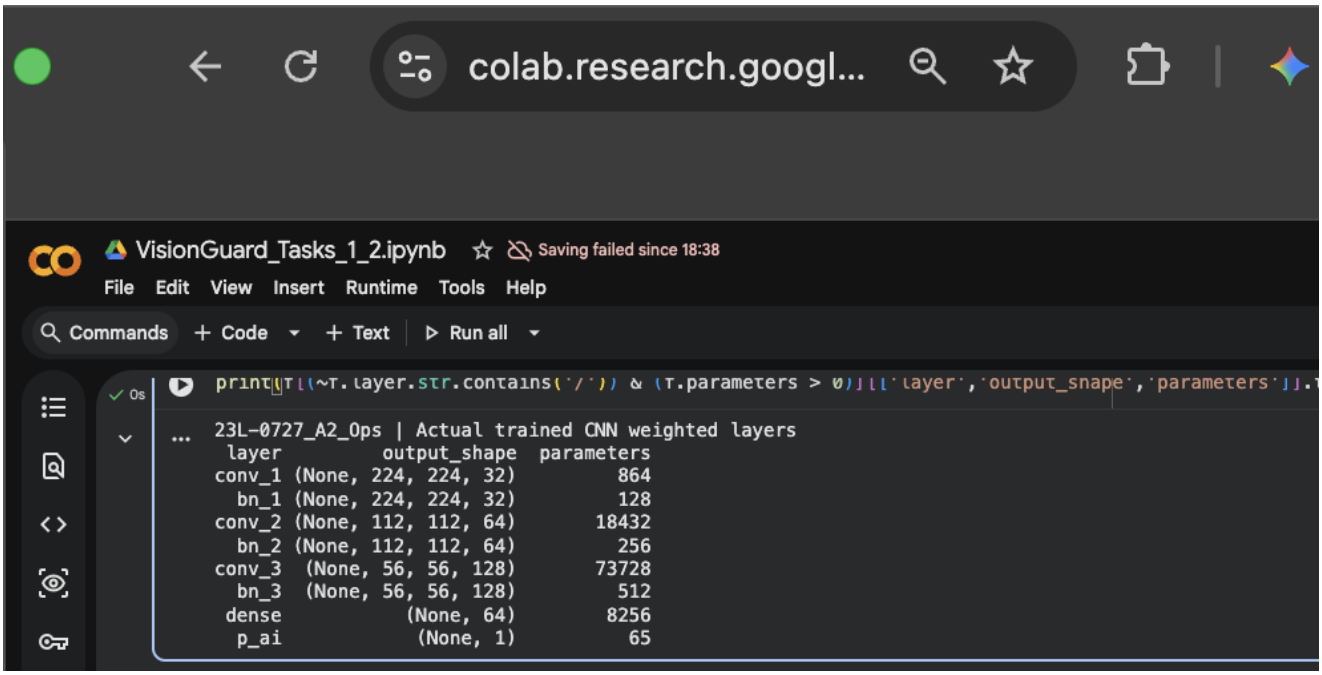


Figure 11. Executed weighted-layer excerpt of the scratch CNN, with output shapes and parameter counts.

2.3 - Transfer learning in two stages

MobileNetV2 uses ImageNet pretrained weights with include_top=False. The added classifier consists of global average pooling, dropout 0.3, and one sigmoid. Embedded rescaling maps 0-255 pixels to [-1,1], matching MobileNetV2 preprocessing. ImageNet features provide a useful initialization, while the authenticity task differs from the original object classification task.

Stage 1 freezes the backbone and trains the head with Adam at 0.001. Stage 2 unfreezes only the last block's weighted non-BatchNorm layers and the terminal convolution, then recompiles with learning rate 0.00001 (100 times smaller). BatchNorm remains frozen and the backbone is called in inference mode to preserve its moving statistics.

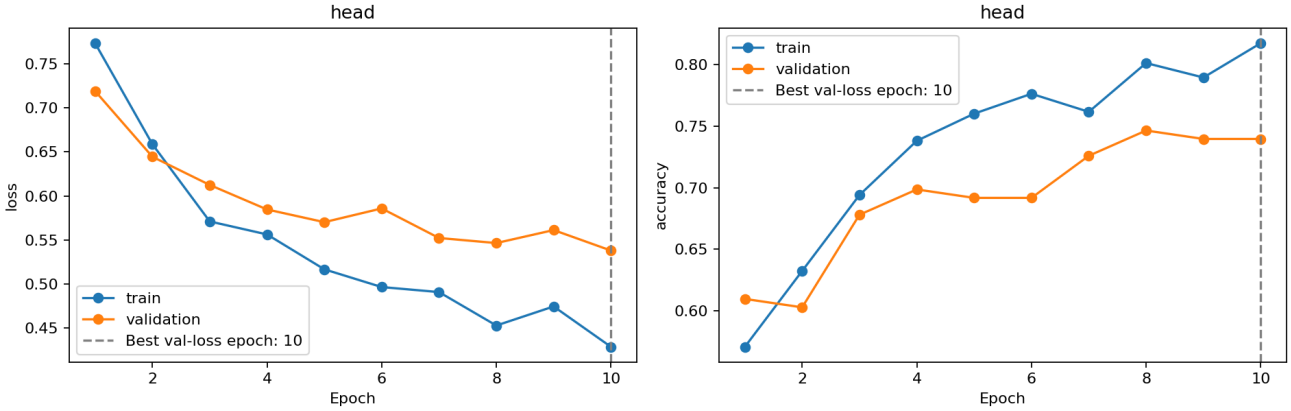


Figure 12. head: loss and accuracy curves, with minimum validation-loss epoch marked.

head trainable weighted layers: p_ai

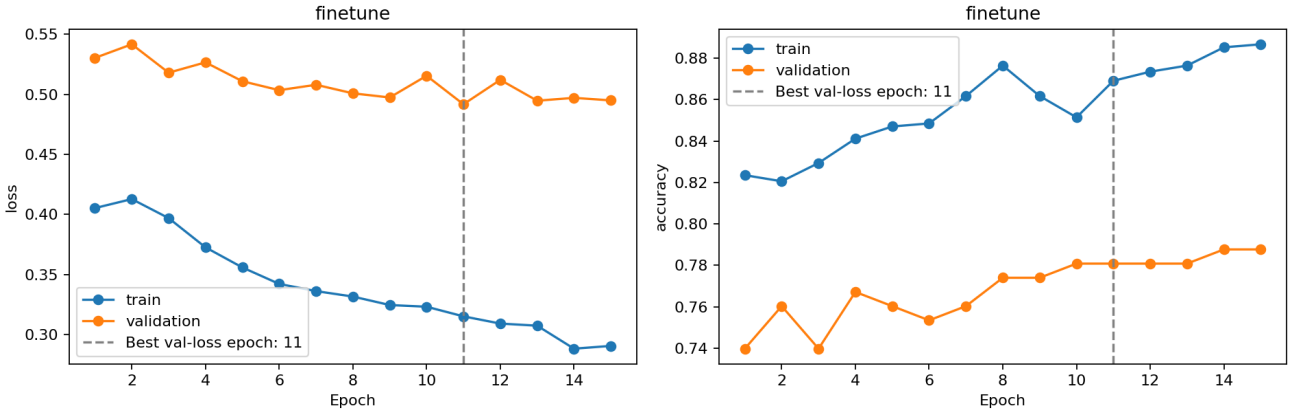


Figure 13. finetune: loss and accuracy curves, with minimum validation-loss epoch marked.

finetune trainable weighted layers: mobilenetv2_1.00_224/block_16_expand, mobilenetv2_1.00_224/block_16_depthwise, mobilenetv2_1.00_224/block_16_project, mobilenetv2_1.00_224/Conv_1, p_ai

The best validation loss fell from 0.53795 for the frozen head to 0.49179 after last-block adaptation. This supports useful adaptation on this validation split, while the later tuner winner achieved 0.47383. The frozen-head best epoch was the budget limit (10), so early stopping did not establish convergence for that stage.

2.4 - Training discipline and checkpoints

Python, NumPy, and TensorFlow seeds are fixed to 42, and TensorFlow deterministic operations are enabled. Exact reproducibility still depends on hardware, CUDA, and package versions. A checkpoint is saved only when validation loss improves. Early stopping uses patience 4 and restores best weights; ReduceLROnPlateau halves the learning rate after 2 stagnant epochs with minimum 1e-7.

Stage	Epoch budget	Epochs run	Best epoch	Best validation loss
baseline	20	10	6	0.66881
head	10	10	10	0.53795
finetune	15	15	11	0.49179

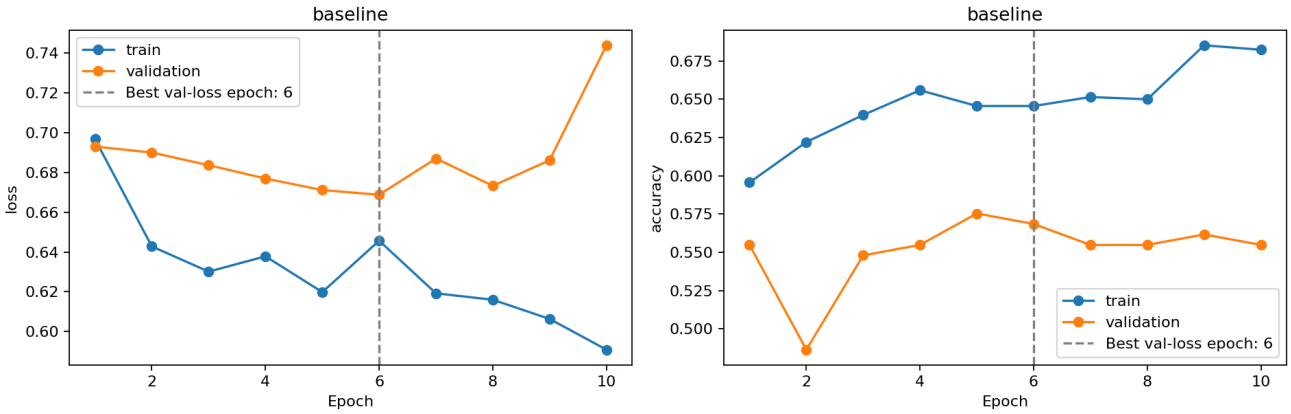


Figure 14. Scratch CNN training curves, with the selected validation-loss checkpoint marked.

Scratch training loss generally decreases, while validation loss improves only to epoch 6 and worsens later. Training accuracy rises above validation accuracy, which stays near the class-majority level. This gap is consistent with overfitting or dataset mismatch; the curves alone cannot identify its cause.

The transfer candidates (frozen head, last block, and tuner winner) were compared by validation loss before test access. The selected candidate was **tuned**. Thresholds and split hashes were saved in selection_before_test.json, then an exclusive test-start marker prevented a second final evaluation in this run directory.

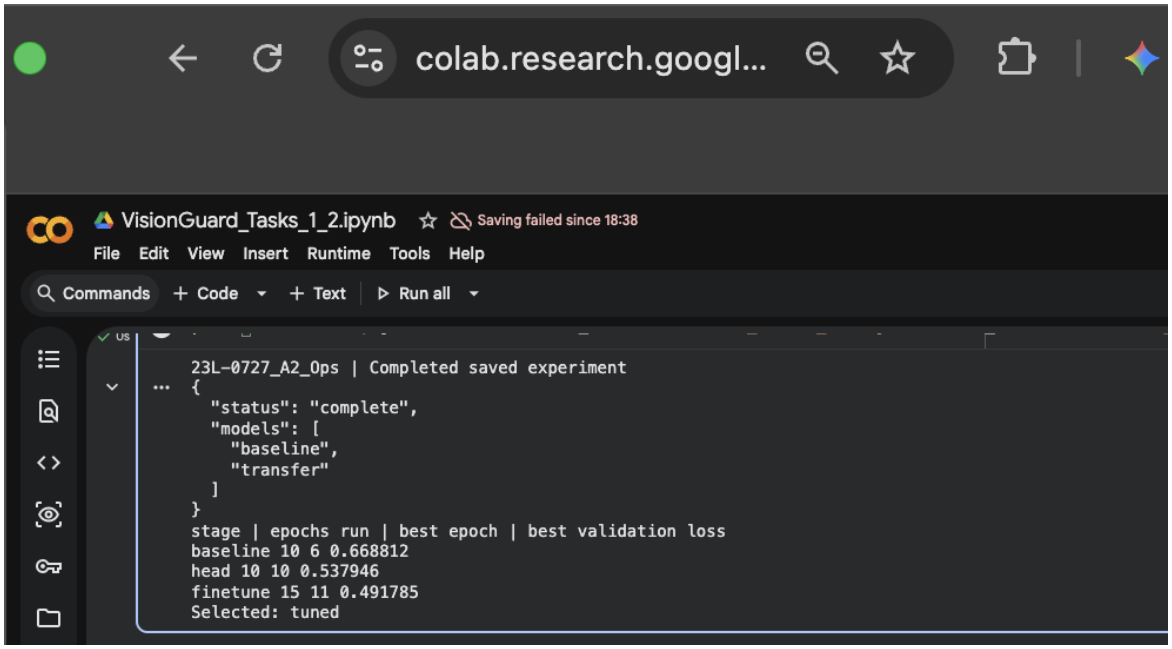


Figure 15. Actual completed experiment evidence and stage results.

2.5 - Held-out evaluation and decision threshold

AI is the positive class. A real photograph incorrectly called fake can damage trust, so the predeclared validation objective is 2 x false positives + false negatives. This 2:1 ratio is a product judgment, not a measured business cost. The algorithm checks every attainable validation boundary and chooses the higher threshold when costs tie. Test labels never influence this selection.

Model	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
baseline	0.65103	0.4932	0.6667	0.1500	0.2449	0.6672
transfer	0.68069	0.7055	0.7761	0.6500	0.7075	0.8377

Model	True real (TN)	Real called AI (FP)	AI missed (FN)	True AI (TP)
baseline	60	6	68	12
transfer	51	15	28	52

baseline: 6 of 66 real images were called AI, and 68 of 80 AI images were missed. Precision measures correctness of AI accusations; recall measures the fraction of AI images caught. ROC-AUC uses continuous scores and does not depend on the selected threshold.

transfer: 15 of 66 real images were called AI, and 28 of 80 AI images were missed. Precision measures correctness of AI accusations; recall measures the fraction of AI images caught. ROC-AUC uses continuous scores and does not depend on the selected threshold.

The tuned transfer model is the preferred model from pre-test validation loss (0.47383 versus 0.49179 for last-block fine-tuning and 0.53795 for the frozen head). The held-out comparison also favors transfer: F1 is 0.7075 versus 0.2449 and ROC-AUC is 0.8377 versus 0.6672. However, it makes more false accusations (15 versus 6), so this preference is not a claim that every error type improves. The scratch threshold is conservative and misses 68 of 80 AI images; its training/validation gap and weak validation discrimination limit its usefulness. Test weighted cost is 58 for transfer versus 80 for scratch under the declared 2:1 rule; these are retrospective comparisons, and later validation-only tracking demonstrations did not change the champion.

The 146-image test set is small and comes from the same collection as training. These are point estimates without a new-source or new-generator guarantee. Source artifacts, label uncertainty, compression, and unseen generators limit generalization; the detector should support review rather than establish authenticity.

2.5 - Executed metric evidence

Figure 16 records the final saved test results in the actual Colab notebook. The report tables use the unrounded JSON values, whereas this compact display rounds to four decimal places. Confusion-matrix counts preserve the exact errors behind precision and recall; all values refer to the same 146 held-out images.

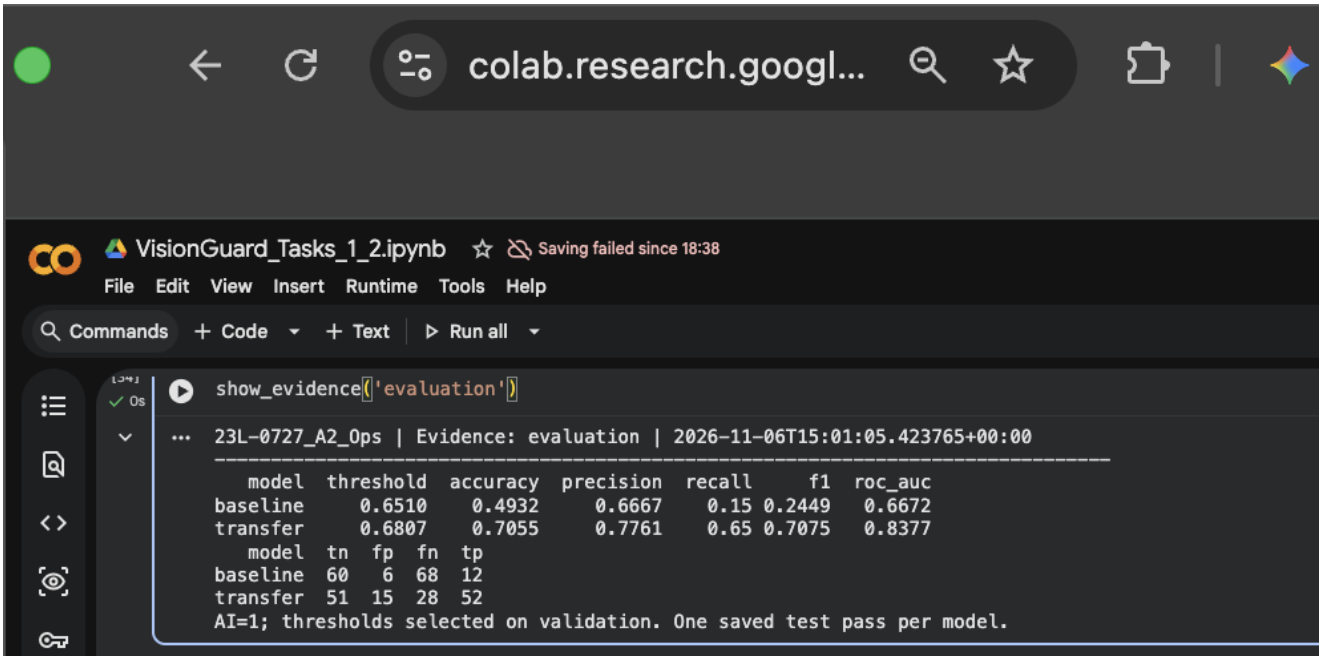
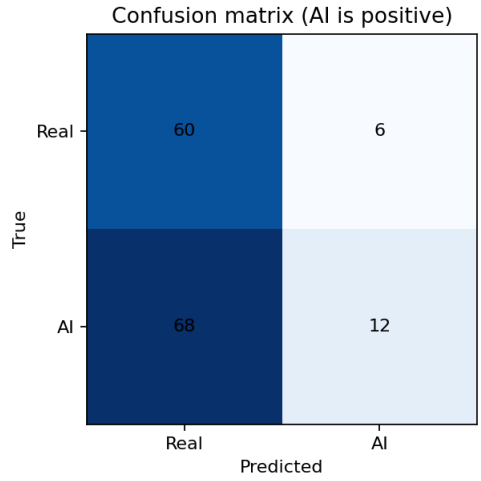
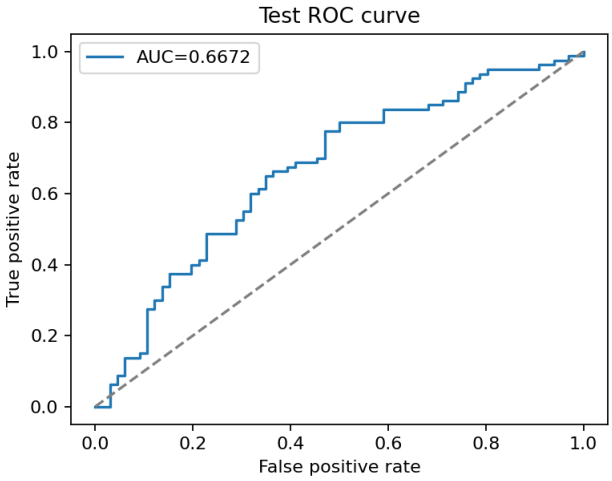


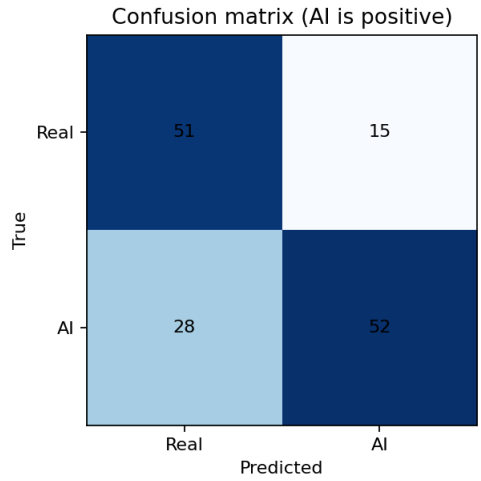
Figure 16. Executed test metric comparison with the project context.

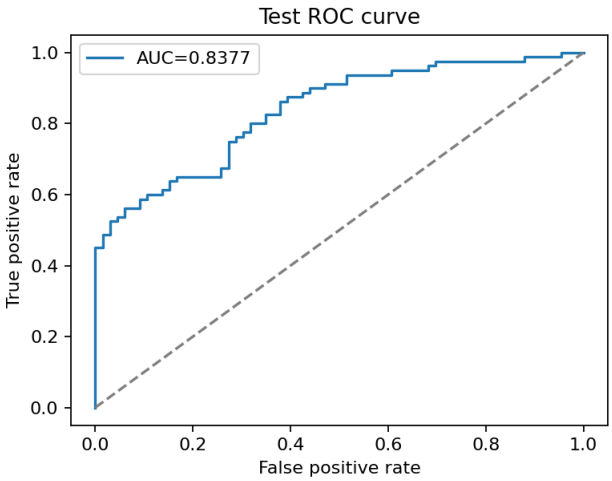
2.5 - Confusion matrices and ROC curves





baseline: confusion matrix (rows=true, columns=prediction) and held-out ROC curve.





transfer: confusion matrix (rows=true, columns=prediction) and held-out ROC curve.

Each final model predicts the test set exactly once. Probabilities, row-level predictions, and labels are saved, and the confusion matrix, ROC, metrics, and error grids are derived from that same pass. Accuracy is (TP+TN)/N; precision=TP/(TP+FP); recall=TP/(TP+FN); F1 is their harmonic mean. The champion was not retrained after the held-out metrics were exposed. Later tracking labs train separate validation-only diagnostic heads and never change the selected champion.

2.6 - Error analysis: transfer

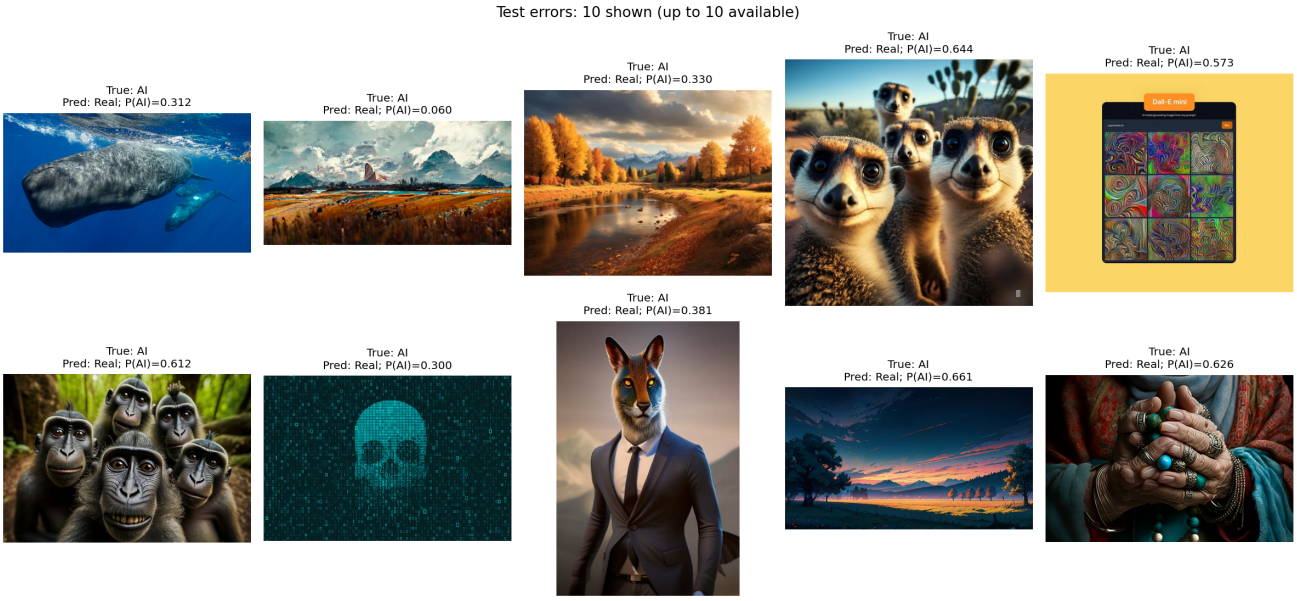


Figure 17. First ten available transfer errors in manifest order; true and predicted labels and AI scores are shown.

Image	Visible observation	Hypothesis and check
1	A whale underwater has plausible lighting and water texture.	ImageNet wildlife features may favor a real-like score; test matched unseen synthetic/real underwater images.
2	A broad colorful meadow is framed by snow-covered mountains.	Natural scene semantics may hide generation artifacts at 224 px; check a separate high-resolution landscape set.
3	Orange autumn trees and a reflective river fill a landscape.	Texture and lighting resemble photography; compare independent real/generated autumn scenes with controlled compression.
4	The foreground animals have large eyes and close-up fur texture.	Recognizable animal fur may dominate unusual proportions; compare unseen stylized renders with real animal close-ups.
5	A yellow page surrounds a Dall-E mini interface and a nine-image grid.	The surrounding screenshot may dilute the small generated tiles; compare tile crops with full screenshots on new examples.
6	Five primate-like faces cluster in front of a green background.	Familiar faces may outweigh stylized facial geometry; test independent generated/real primate portraits.
7	A turquoise skull is drawn using small binary digits on a dark field.	Graphic art also occurs in the real folder; inspect label provenance and evaluate independently curated digital artwork.
8	An upright animal-headed figure wears a suit and tie.	Real-folder illustrations may overlap anthropomorphic art; compare independent composite/illustration categories.
9	A stylized sunset landscape uses broad dark-blue and orange color bands.	Painting-like real data may share this visual style; test a separately sourced illustration holdout.
10	A close-up shows many fingers, rings, beads, and patterned fabrics.	Local hand geometry may be missed by global pooling; inspect high-resolution hand regions on new examples.

Images are numbered left-to-right, top-to-bottom. Sorted paths place AI images first, so these first ten available errors are all false negatives; the confusion matrices cover both error types. These are visual observations and testable hypotheses, not proven causes. Proposed diagnostics were not used to retune the model or threshold.

2.7 - Keras Tuner: Fashion-MNIST toy

The toy uses a tiny flattened Fashion-MNIST classifier with one hidden ReLU layer and a 10-class softmax. A fixed stratified subset of 10,000 training and 2,000 validation images is taken from the built-in training partition; its test partition remains unused. Pixels are scaled to [0,1], with sparse categorical cross entropy for integer class labels.

Random search explores 32/64/128 hidden units and Adam learning rates 0.01/0.001/0.0001. Eight trials run up to five epochs with early stopping (patience 2). The objective is minimum validation loss, so lower is better.

Trial	Status	Units	Learning rate	Validation loss
1	COMPLETED	128	0.001	0.45032
4	COMPLETED	64	0.001	0.45060
3	COMPLETED	32	0.001	0.45396
0	COMPLETED	64	0.01	0.48048
5	COMPLETED	32	0.01	0.48549
2	COMPLETED	128	0.01	0.49178
6	COMPLETED	128	0.0001	0.52975
7	COMPLETED	64	0.0001	0.57617

Best toy settings: **{'units': 128, 'learning_rate': 0.001}**. This lab demonstrates tuning mechanics on an unrelated built-in dataset; its score is not VisionGuard performance.

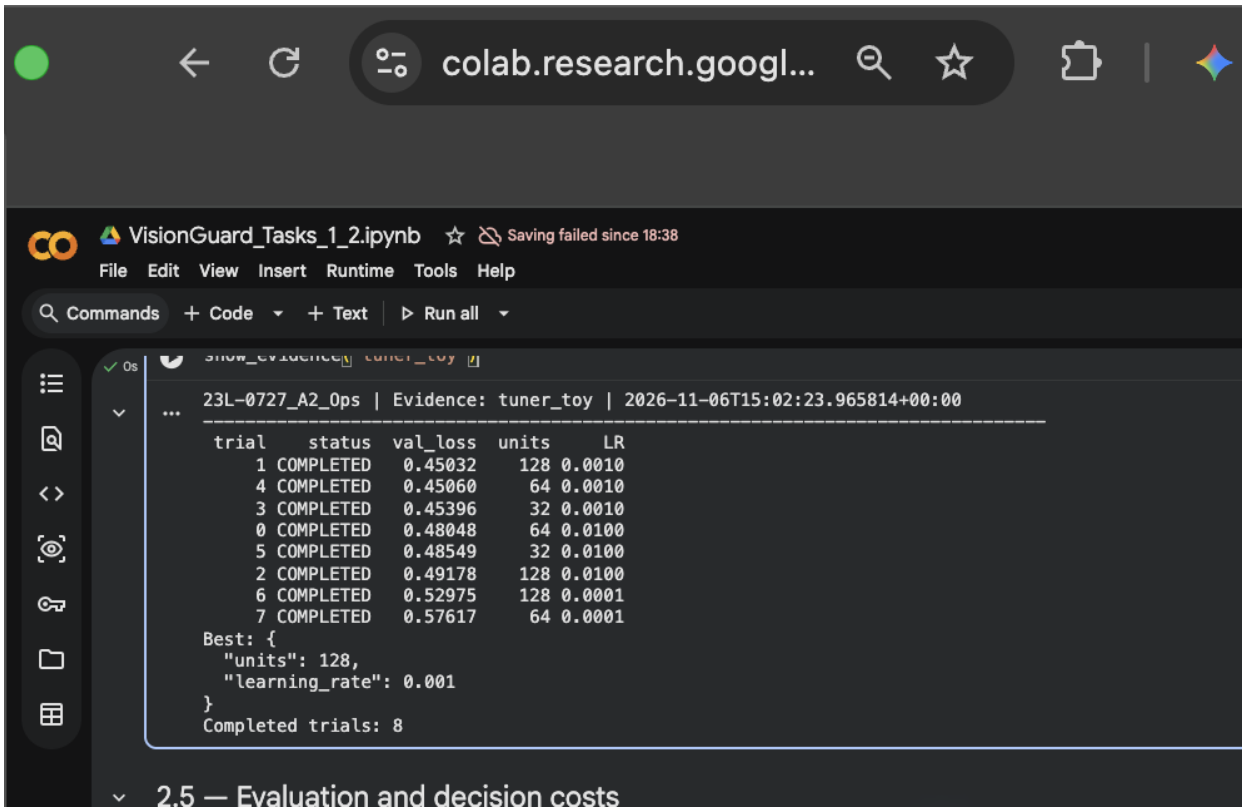


Figure 18. Actual Fashion-MNIST toy search results.

2.7 - Applied VisionGuard tuning

Every trial starts from the same trained frozen-head checkpoint. Random search varies learning rate {1e-5, 3e-5, 1e-4}, head dropout {0.2, 0.4, 0.5}, and {1, 2, 4} unfrozen weighted layers within the final MobileNetV2 block/terminal convolution. Earlier layers and BatchNorm remain frozen. Each of eight trials runs up to five epochs with early stopping and learning-rate reduction.

Trial	Status	LR	Dropout	Unfrozen	Val loss
4	COMPLETED	0.0001	0.2	2	0.47383
6	COMPLETED	0.0001	0.2	1	0.48911
1	COMPLETED	3e-05	0.2	4	0.49624
2	COMPLETED	0.0001	0.5	1	0.50121
5	COMPLETED	3e-05	0.4	2	0.51677
3	COMPLETED	1e-05	0.2	2	0.51849
0	COMPLETED	3e-05	0.4	1	0.52119
7	COMPLETED	1e-05	0.5	4	0.52495

Best applied settings: **{'dropout': 0.2, 'unfrozen_layers': 2, 'learning_rate': 0.0001}**. The actual winning checkpoint is retained; the final transfer candidate is selected against the head and standalone fine-tune candidates using validation loss.

2.7 - Applied tuner evidence and interpretation

Figure 19 confirms that all eight applied random-search trials completed and records the winning settings. Trial IDs are zero-based, so winning trial 4 is the fifth trial. These are validation results; test data did not choose the hyperparameters.

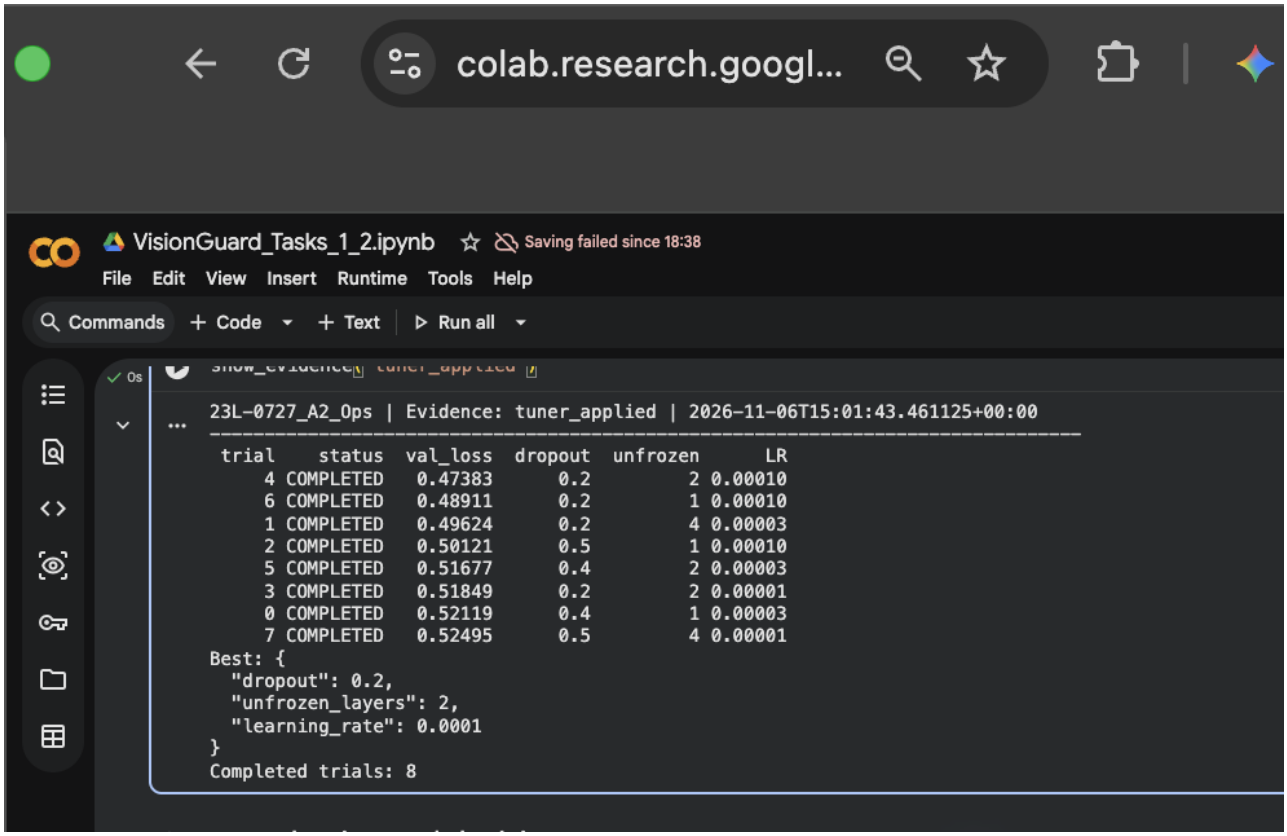


Figure 19. Eight completed applied trials and winning settings.

Parameter	Range of marginal mean validation losses
learning_rate	0.033671
dropout	0.024562
unfrozen_layers	0.007562

learning_rate has the widest observed marginal mean-loss range, and thus the strongest exploratory association in these trials. This is not causal importance: multiple settings change together, each value has few observations, and optimization is stochastic. A controlled one-factor comparison would be needed to isolate effects.

The marginal mean validation loss was 0.52172 at 1e-5 (2 trials), 0.51140 at 3e-5 (3), and 0.48805 at 1e-4 (3). Larger updates may have adapted the pretrained final layers faster within the short five-epoch budget. This is a hypothesis about the observed association, not proof that 1e-4 is universally best.

Choosing the minimum of many validation scores can select random favorable noise and overfit the validation split. The test set is consequently opened only after candidate and threshold selection. Further model changes after this report would require a fresh held-out set for an unbiased final check.

Reproduction, troubleshooting, and references

The source repository contains the audit, pipeline/models, tuning, threshold/evaluation code, and standalone notebook. The evidence bundle includes the exact environment, split manifests, source hashes, curves, checkpoints, tuner state, and saved predictions. The report builder reads these outputs and refuses to build without COMPLETE.json and at least eight completed applied trials.

Observed issue	Resolution and evidence
PNG decoding failure	Lossless re-encoding from the audited original; 972/972 pixel hashes verified, one file repaired.
Slow repeated full-resolution decode	Cache resized deterministic inputs before per-epoch shuffle; augmentation remains dynamic.
Google Drive storage quota	Use runtime-local output storage and download evidence/checkpoints. Existing Drive content was not deleted.

For a new run: create a virtual environment; install requirements; audit the originals; verify labels and split hashes; run the PNG integrity repair; execute scripts.run_training with a new output directory. Use the provided unchanged settings for a pre-test resume. After test access, read saved predictions rather than repeating model selection.

Assignment dataset: https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images

TensorFlow transfer learning: https://www.tensorflow.org/guide/keras/transfer_learning

Keras Tuner getting started: https://keras.io/keras_tuner/getting_started/

DVC remote storage: https://dvc.org/doc/user-guide/data-management/remote-storage

Task 3 - Experiment tracking and model registry

3.1 - Local MLflow tool lab

A local MLflow server uses a SQLite backend and a separate artifact directory. Two actual Fashion-MNIST runs change only Adam learning rate (0.01 versus 0.001), using 1,024 training and 256 validation examples, a 32-unit dense layer, batch 64, and three epochs. This small tool lab is distinct from the Task 2 Keras Tuner lab.

Learning rate	Validation loss	Validation accuracy	Run ID
0.01	0.78746	0.6914	b554c92b899d45188b475499184df722
0.001	1.02291	0.6289	40bee3fae20341669ed63ba22e3b24f2

For this fixed three-epoch budget, 0.01 gives lower validation loss and higher validation accuracy. This does not establish the best learning rate for another architecture, seed, or training duration. Per-epoch metrics, parameters, a loss plot, and the saved model are logged.

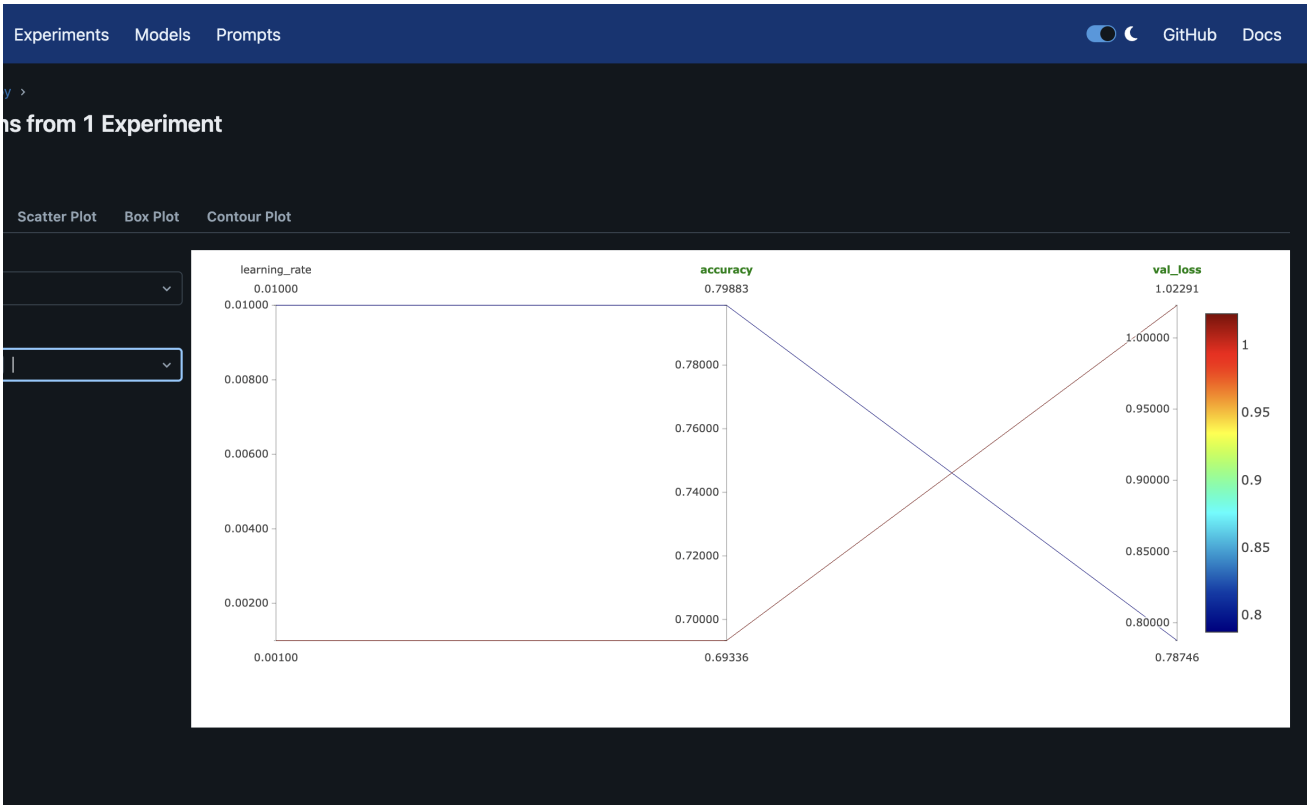


Figure 20. Local MLflow UI: two completed toy runs, with experiment context.

3.2 - DagsHub remote tracking

The same tool lab was run again against HayyanHaider/23L-0727_A2_Ops.mlflow. Credentials are read from the ignored environment file. The remote service stores real runs and artifacts; local JSON files are supporting exports. Random seeds and data version tags make provenance visible.

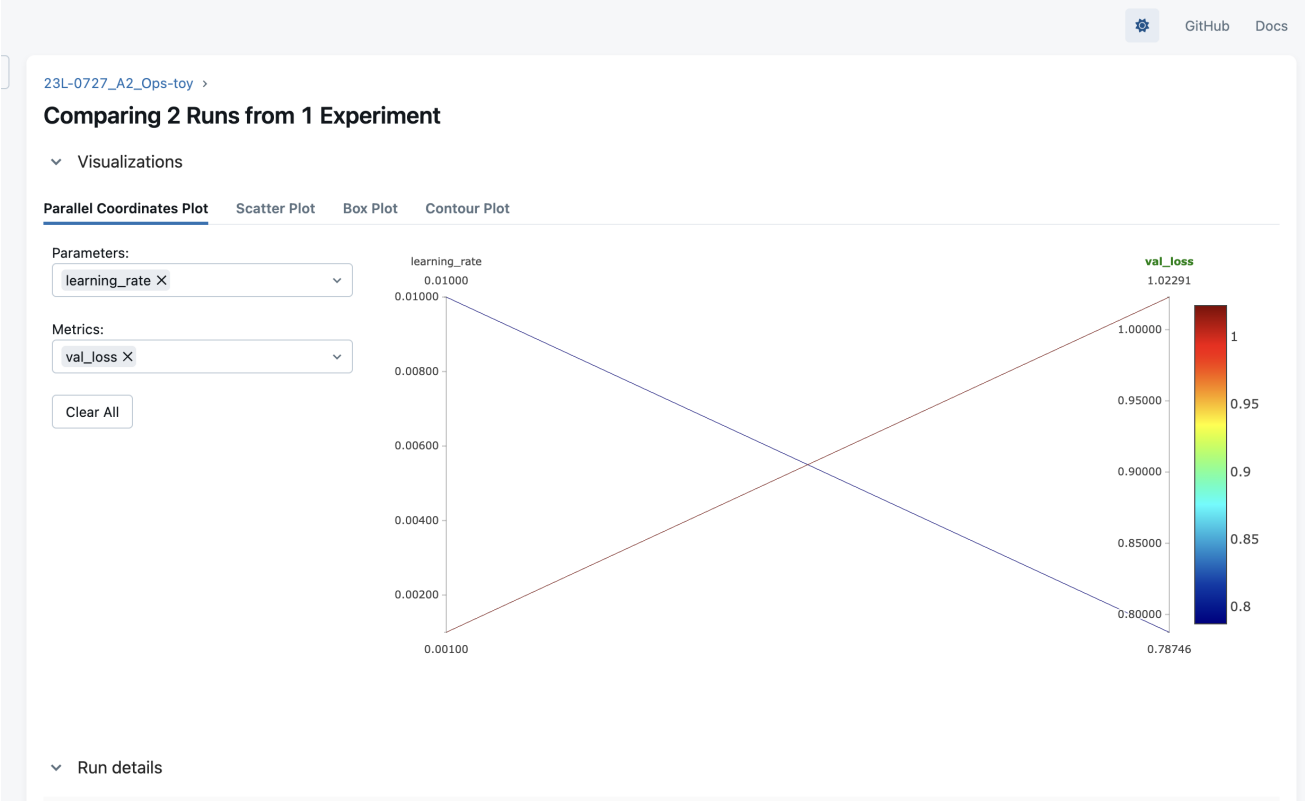


Figure 21. DagsHub-hosted MLflow toy experiment: two actual remote runs.

3.3 - Manual callbacks and TensorFlow autologging

MlflowEpochLogger sends every epoch metric with its epoch step. The separate autolog run calls mlflow.tensorflow.autolog before fit; the manual callback is omitted for that run. Both methods record training and validation behavior. Run tags include the source commit, DVC pointer hash, seed, project, and logging method. Model files, actual learning curves, split manifests, and the exact tracking environment lock accompany the runs.

The original Colab experiments are also uploaded with explicit-historical-import tags. Their saved epoch CSVs, original stage settings, curves, and checkpoints are preserved. Those imports are not represented as live autologging. The six additional training runs below are actual live instrumentation demonstrations.

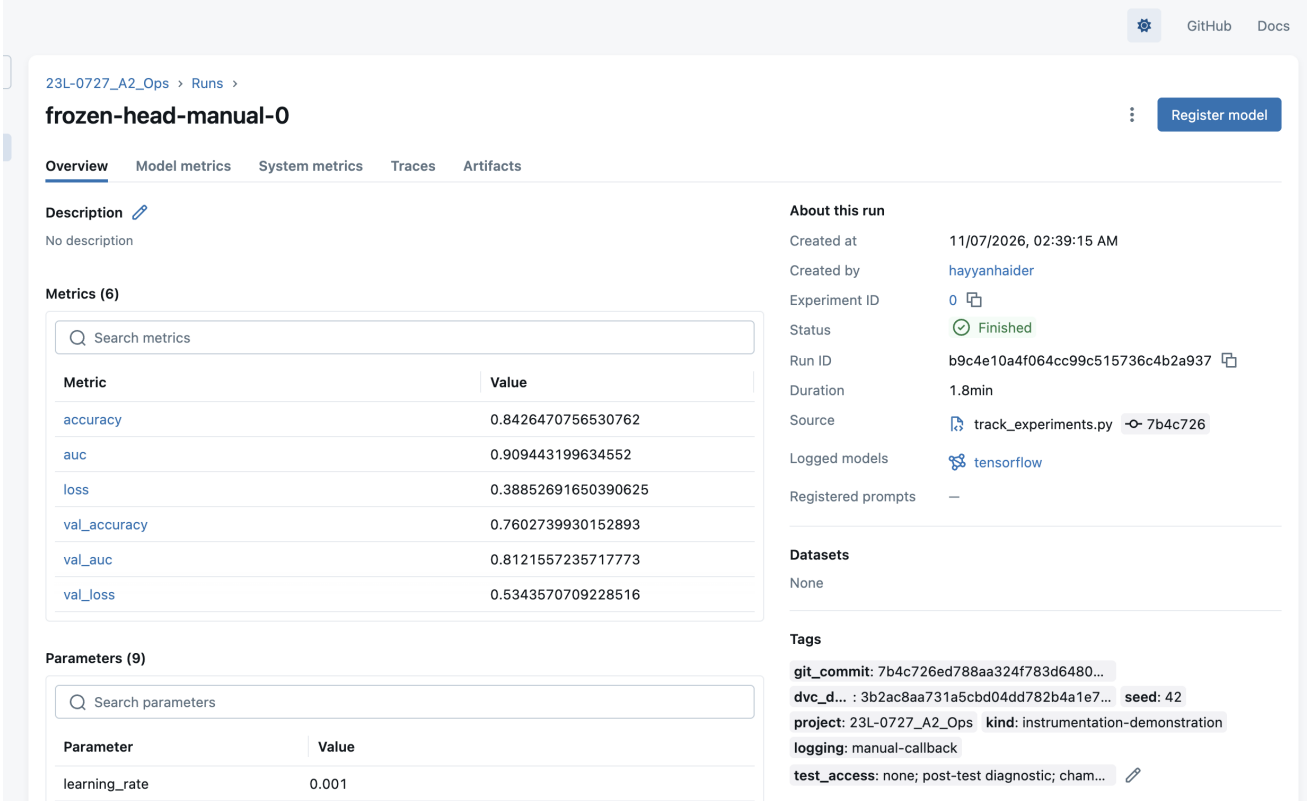


Figure 22. A live manually logged run: parameters, tags, and recorded metric values.

3.3 - TensorFlow autologging evidence

This separate live run uses TensorFlow autologging with the manual epoch callback omitted. Its five training and validation metric steps were verified through the remote API. Framework-generated optimizer parameters and validation metrics accompany the configured learning rate and dropout.

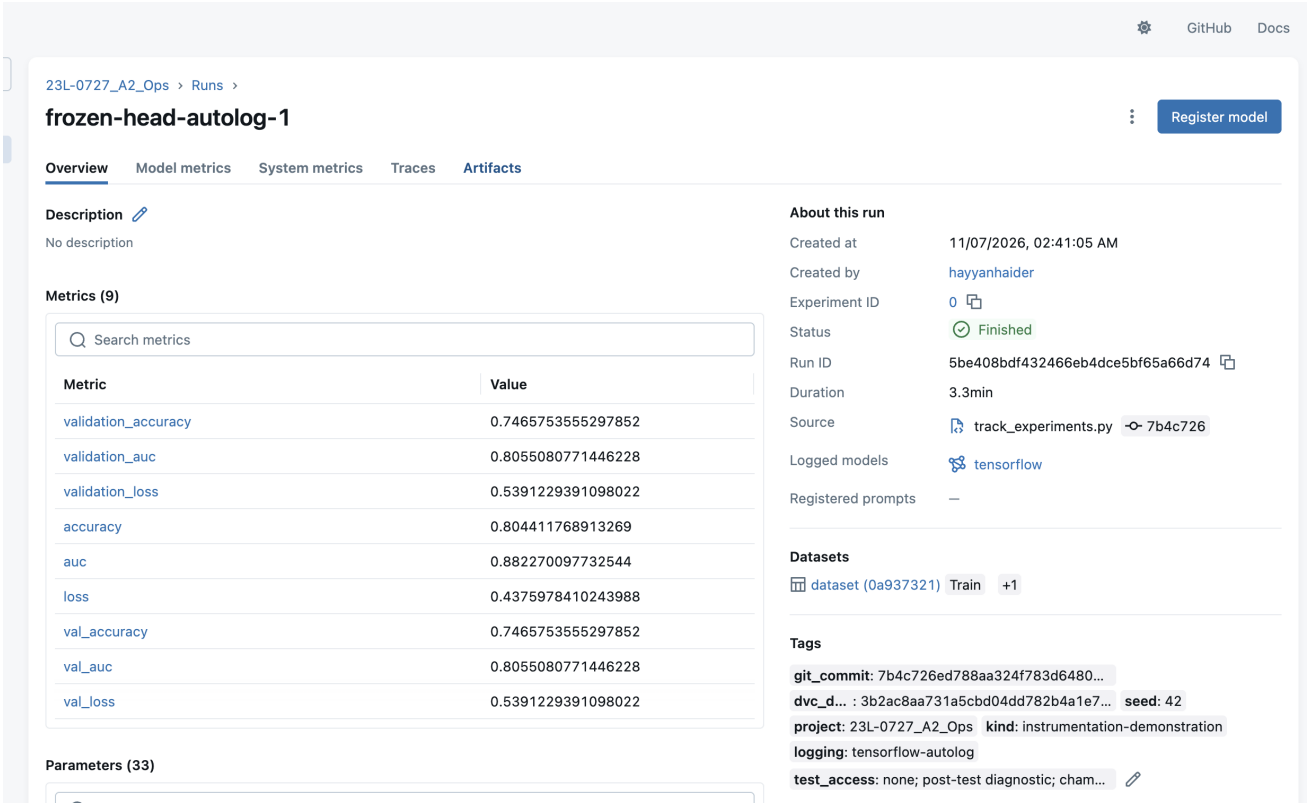


Figure 23. The TensorFlow autolog run: framework-generated epoch metrics and parameters.

3.4 - Six controlled validation-only runs

The original held-out evaluation had already happened. To avoid selecting a new model against exposed test results, all six diagnostic runs use only the original 680 training and 146 validation examples. Each initializes the same original frozen-head checkpoint, caches its 1,280-dimensional MobileNetV2 features, and trains a dense head for five epochs, batch 32. Backbone and BatchNorm stay frozen, with augmentation disabled. Learning rate or dropout changes one factor at a time relative to the baseline. No test inference is performed, and none of these runs is promoted.

Method	LR	Dropout	Train loss	Val loss	Val accuracy
manual	0.001	0.3	0.3885	0.5344	0.7603
autolog	0.0001	0.3	0.4376	0.5391	0.7466
manual	0.01	0.3	0.3323	0.7244	0.7329
manual	0.001	0.0	0.3151	0.5360	0.7740
manual	0.001	0.5	0.4490	0.5346	0.7671
manual	0.001	0.7	0.5639	0.5374	0.7466

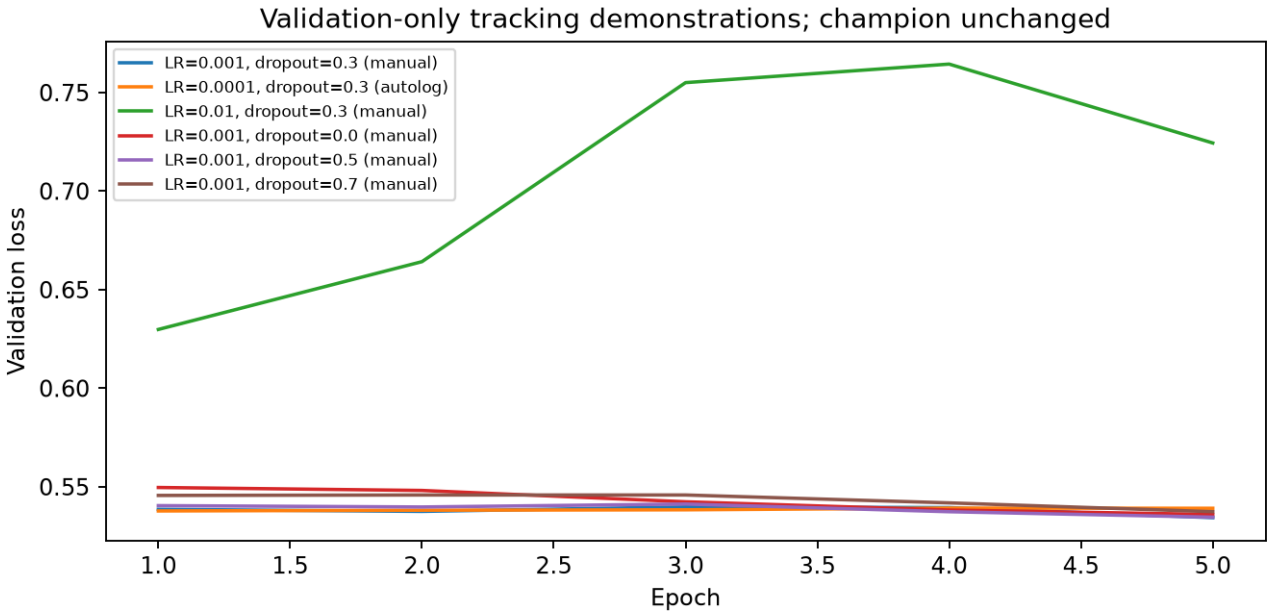


Figure 24. Actual remote run histories: validation loss over five epochs.

Increasing learning rate from 0.001 to 0.01 reduces training loss but increases final validation loss to 0.7244, consistent with poorer validation generalization at this budget. The 0.0001 run adapts more slowly. Removing dropout increases training fit, while validation loss stays near the baseline. Dropout 0.5 and 0.7 do not improve final loss here. Small differences near 0.534 are not persuasive without repeated seeds. Original tuned champion loss 0.47383 remains better, but it also adapts backbone layers, so this is not a controlled architecture comparison.

3.4 - Remote six-run comparison

The native remote comparison contains all six live diagnostic runs. The table and loss chart on the preceding page use the actual five-epoch histories and provide the controlled learning-rate/dropout comparison. IDs and full parameter dictionaries are supplied in remote-mlflow-audit.json.

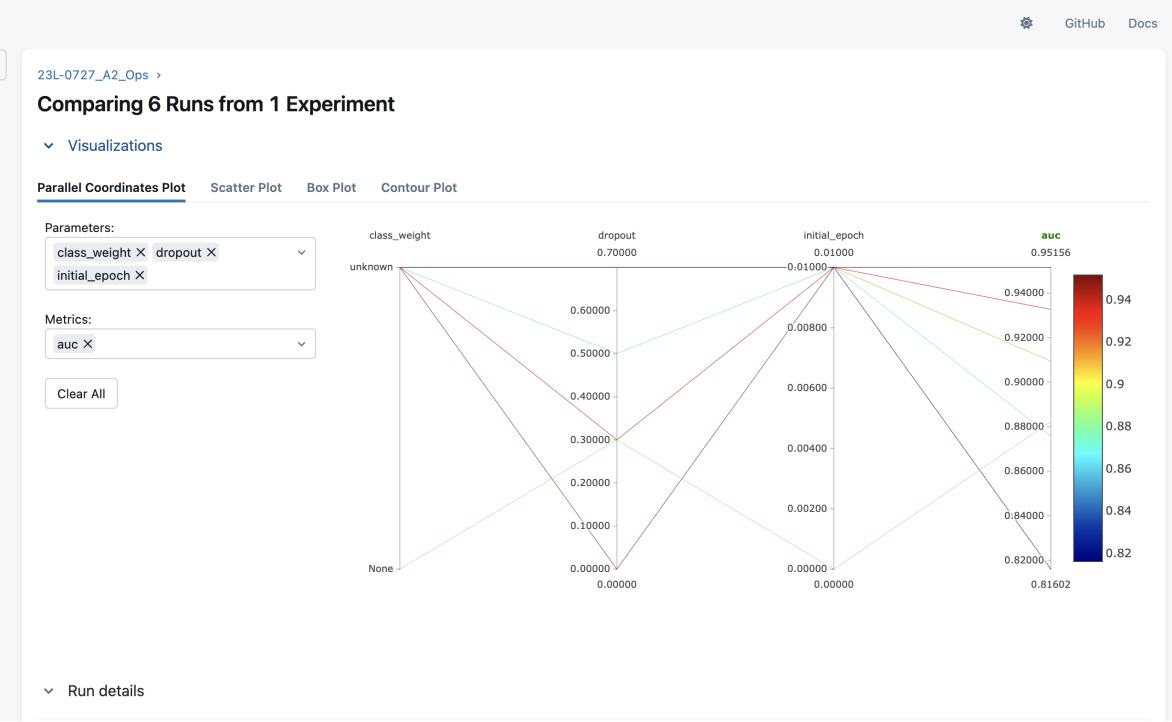


Figure 25. Remote experiment view containing the six distinct diagnostic configurations.

3.5 - Artifacts and model registry

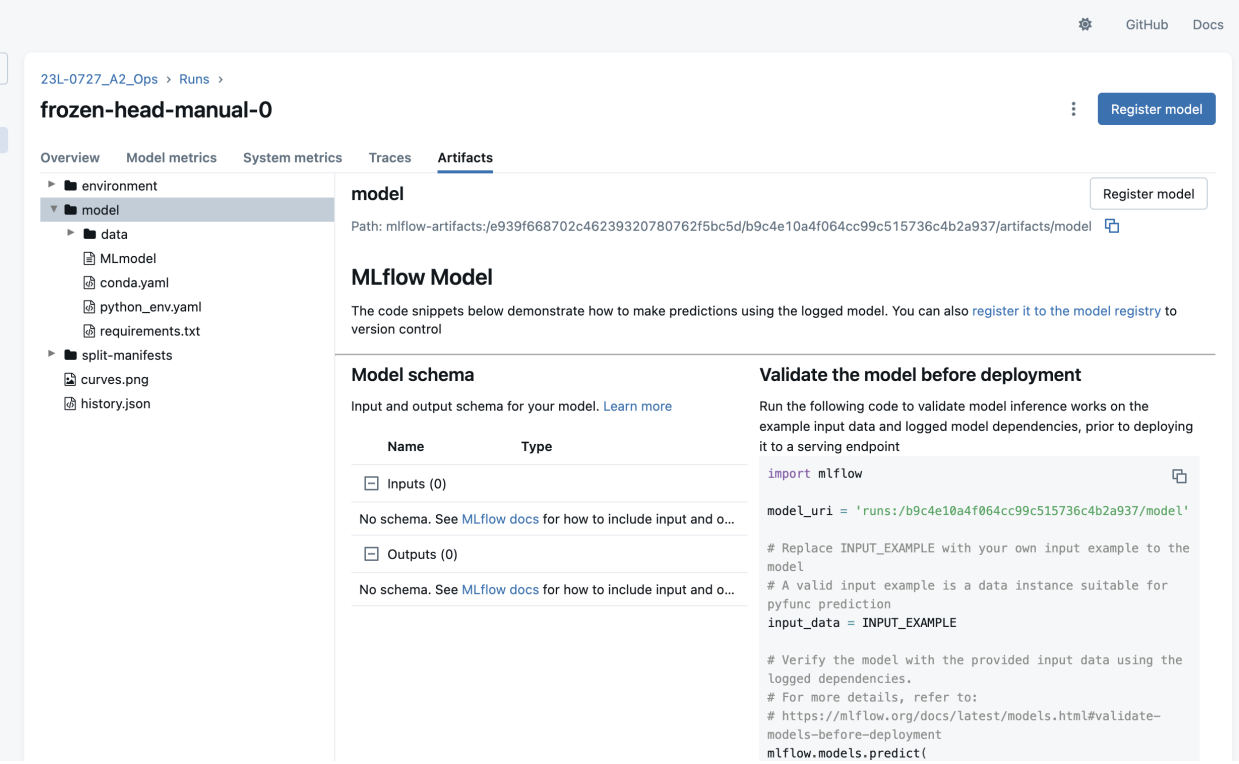


Figure 26. Actual remote artifact browser: model and reproducibility evidence.

The artifact browser confirms stored model files, environment, split manifests, loss curves and epoch histories. Split manifests preserve membership; the DVC version tag identifies dataset metadata, while the environment lock identifies the actual tracking runtime.

3.5 - Registry promotion and alias inference

The registry contains version 1 from the original frozen-head checkpoint and version 2 from the original winning tuned checkpoint. The champion alias resolves to version 2. scripts.predict_by_alias is a separate process: it resolves models:/23L-0727_A2_Ops@champion, loads the model, exports it, and predicts a training example. The registry export is reserialized, so its file hash differs; all 262 weight arrays were verified exactly equal to the original deployment champion. Release provisioning verifies the original file SHA-256 before use.

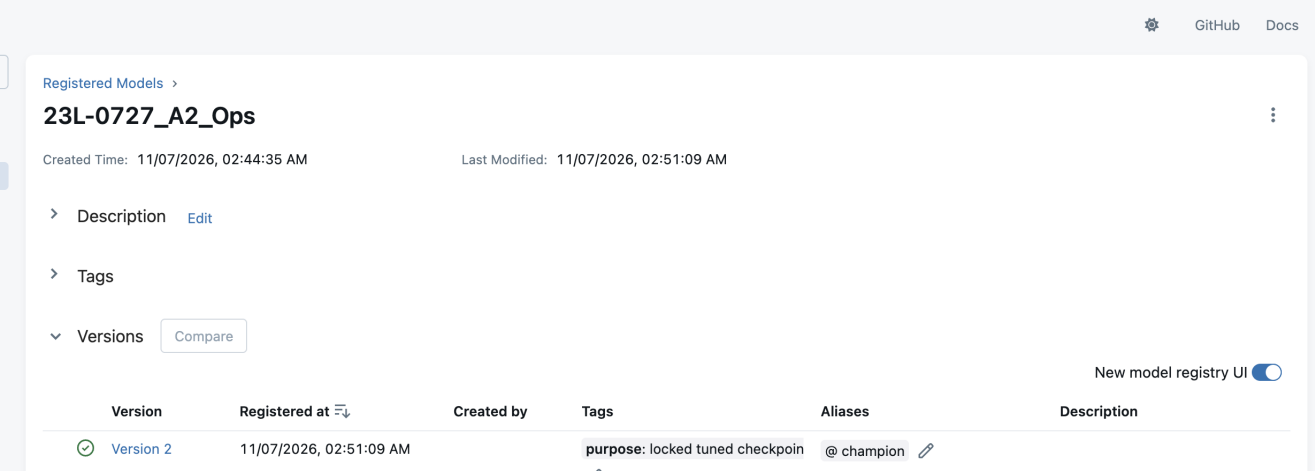


Figure 27. DagsHub model registry: two actual versions and the champion alias.

Check	Observed result
Registry alias	champion -> version 2
Independent alias prediction	Known-real training sample: Real; AI score 0.21879
Registry/release parity	262 weight arrays exactly equal
Remote run count	20 completed runs: 2 toy + 6 live diagnostics + 12 original imports

Task 4 - Flask inference application

4.1 - Model loading and preprocessing

visionguard.inference separates prediction from routes. Each Gunicorn worker loads the champion once at application startup with compile=False, verifies its metadata hash, and warms it up. Requests reuse that predictor. Images are decoded in memory, EXIF orientation is applied, and RGB pixels are resized with aspect-preserving zero padding to 224 x 224 using TensorFlow bilinear antialiasing. Embedded MobileNet normalization maps the resulting 0-255 values to [-1,1]; inference disables training augmentation.

The sigmoid is the AI score. The validation-selected threshold 0.6806868 determines the label. Displayed confidence is the score for the predicted class, with an explicit calibration caveat. It is not a forensic guarantee. Filename sanitization removes path components, and raw uploads are not written to disk.

4.2 - Routes and user flow

Route	Behavior
GET /	Upload form; accepted types and limit
POST /predict	HTML result with decision, score, model version and request ID
POST /api/predict	API-key authenticated multipart request; JSON response
GET /history	Recent persisted records
GET /health	Database readiness, deployment mode, model version, debug state

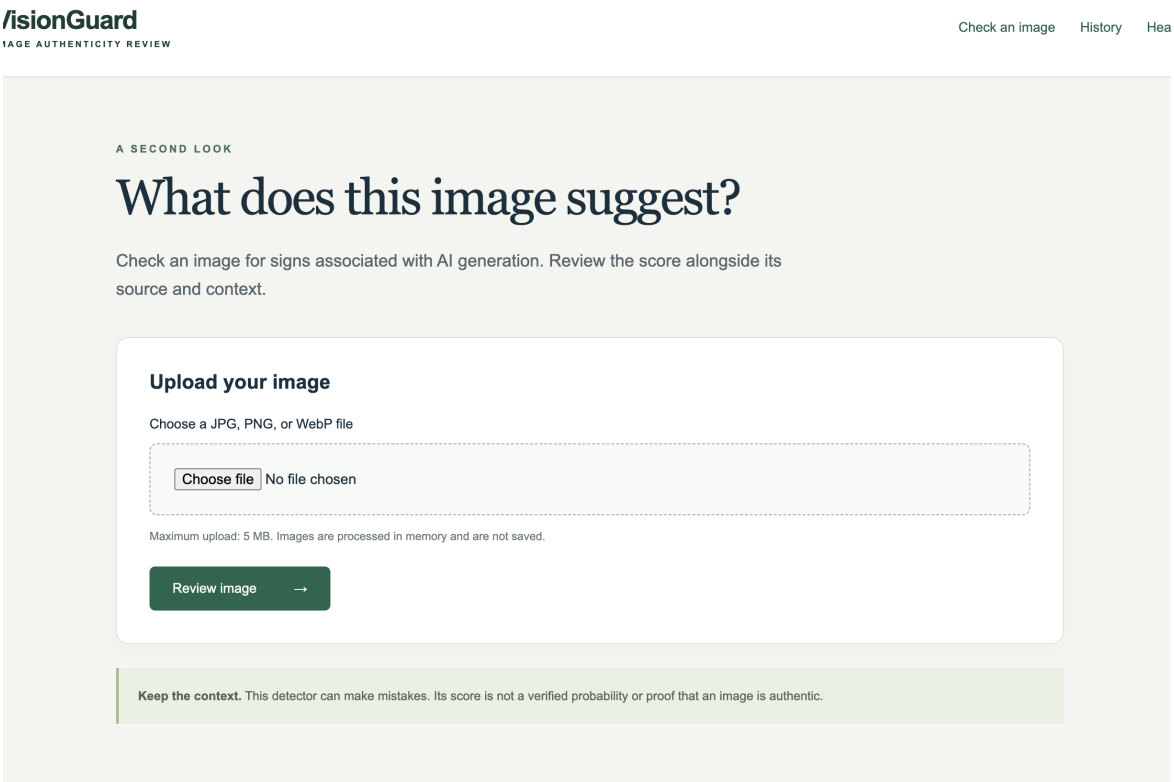


Figure 28. Actual local upload page with project and student context.

4.3 - Results, validation failures, and configuration

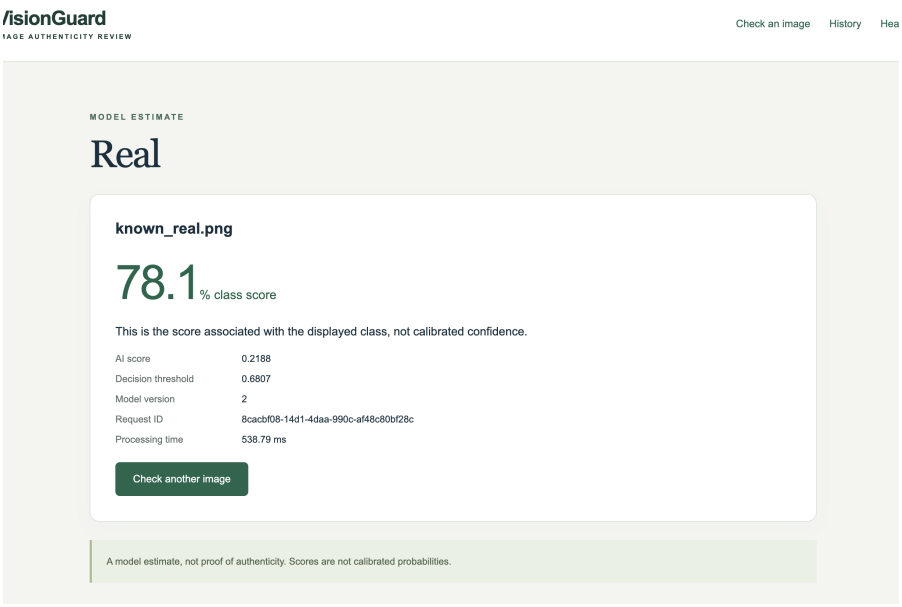


Figure 29. Actual HTML response from resident-model inference; model version and score are visible.

The live HTTP smoke test accepted the real and AI demonstration PNGs. It also exercised an unsupported .exe extension (400), a corrupt PNG (400), and an upload over 5 MB (413). Decoder format is checked as well as extension, so renaming arbitrary bytes to .png is rejected. A decoded-pixel limit prevents very large decompressed images. Errors produce friendly responses without stack traces; unexpected failures are logged and return 503.

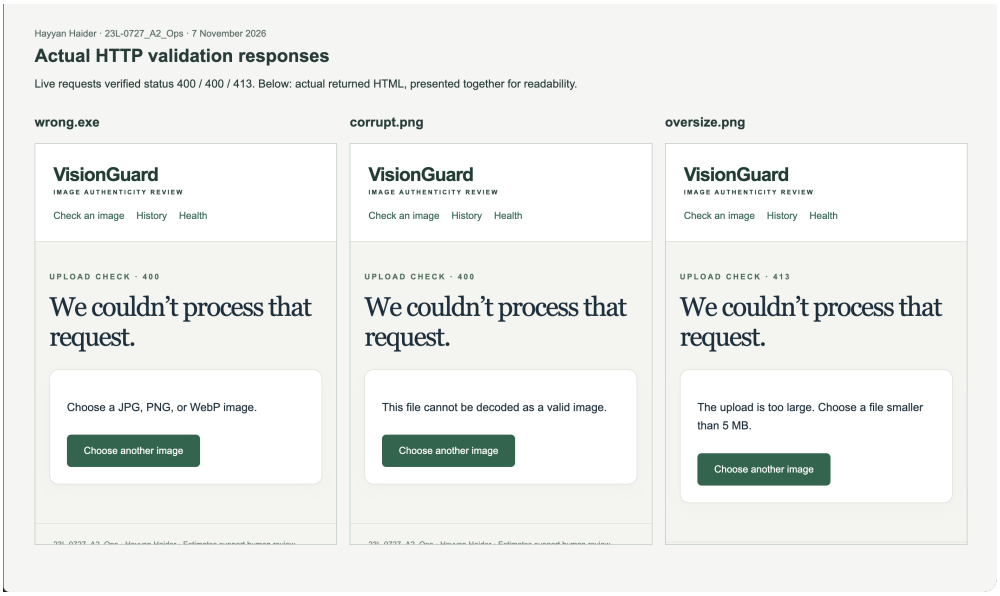


Figure 30. Saved actual HTTP error responses rendered for inspection; all three failures were verified over HTTP.

MODEL_PATH, MODEL_METADATA_PATH, MODEL_THRESHOLD, MAX_UPLOAD_BYTES, DATABASE_URL, INFERENCE_API_URL, and INFERENCE_API_KEY are environment settings. .env.example contains placeholders; the real .env is ignored and excluded from the archive. Flask debug is false and Gunicorn provides the serving process.

Task 5 - Persistent prediction history

SQLAlchemy stores every successful prediction transaction. The production-like Compose configuration uses PostgreSQL 17 on its internal service network. A local SQLite database is available for a standalone run and the Codespace API. A UUID identifies each request; UTC time, sanitized filename, label, AI score, predicted-class confidence, threshold, model version, model hash, and latency preserve what happened and which model produced it. Failed uploads create no prediction rows.

The history route queries recent records rather than maintaining a process-local list. Raw image bytes and API keys are not part of the prediction table. Filenames can still reveal information, so a real production system would need retention and access controls.

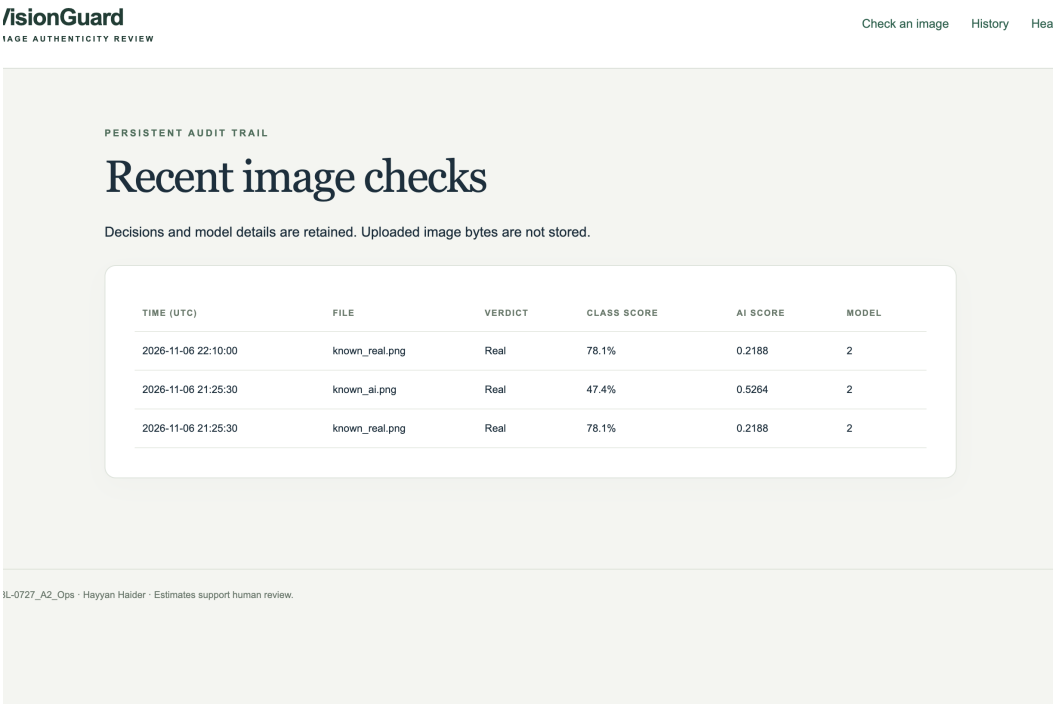


Figure 31. Actual prediction history page, populated by successful requests.

5 - PostgreSQL records and restart persistence

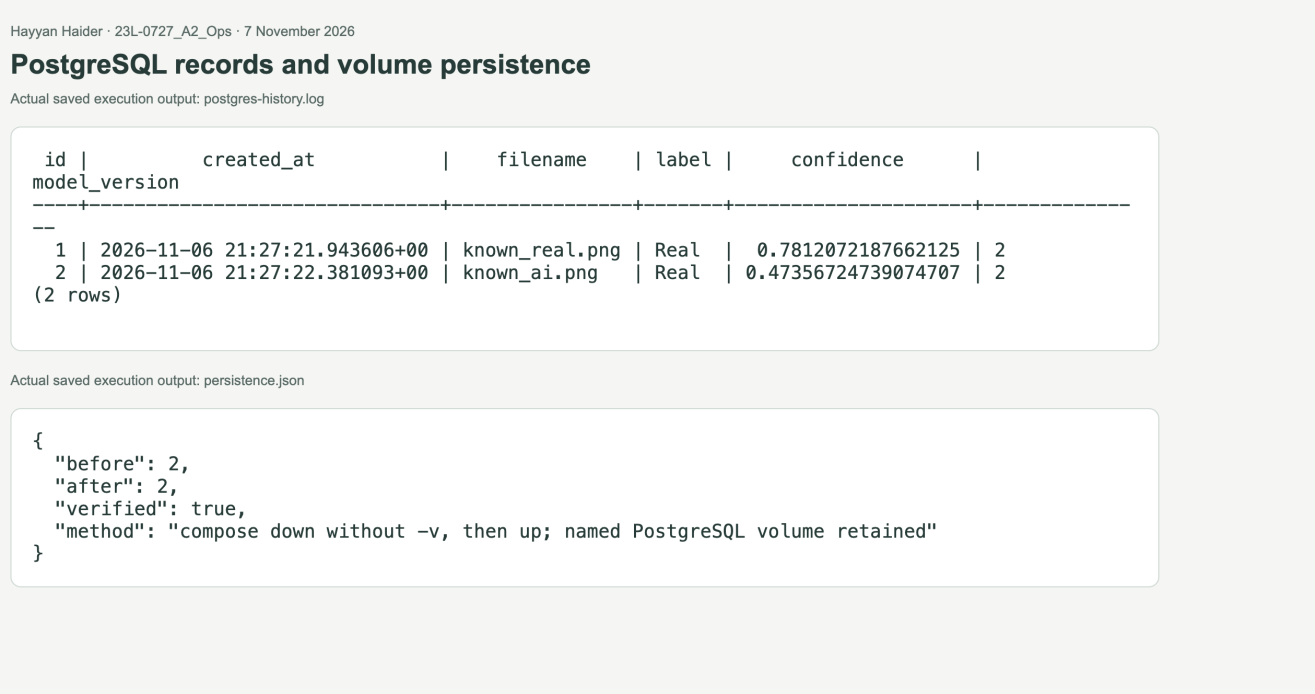


Figure 32. Actual SQL query output and persistence checks from the PostgreSQL container.

The container stack was removed with compose down and started again without deleting volumes. The same two records existed before and after (2 -> 2), confirming persistence in the named PostgreSQL volume. Removing the volume with down -v would intentionally remove that database and is not part of normal restart instructions.

Task 6 - Containers, Compose, and CI/CD

6.1 - Docker warm-up

A disposable BusyBox container demonstrated image pull, run with a published HTTP port, container listing, HTTP access, logs, shell execution, stop, container removal, persistent volume recovery, image removal, and volume cleanup. The transcript records actual commands and their outputs. A file written in the first container remained readable in a second container mounting the same named volume.

Hayyan Haider · 23L-0727_A2_Ops · 7 November 2026

Docker warm-up - execution transcript (1/2)

Actual saved execution output: docker_warmup.log

```
23L-0727_A2_Ops | Docker warm-up | 2026-11-06T21:17:01.790798+00:00

$ docker run -d --name visionguard-warmup -p 127.0.0.1:19080:8080 -v visionguard_warmup_data:/www
busybox:1.37 sh -c echo VisionGuard-23L-0727 > /www/index.html; echo warmup-started; httpd -f -p
8080 -h /www
Unable to find image 'busybox:1.37' locally
1.37: Pulling from library/busybox
af820adcc158: Pulling fs layer
9eac587a9921: Download complete
af820adcc158: Download complete
af820adcc158: Pull complete
Digest: sha256:a7dbc319b1438115a207e11930dc4bb3e643dcb3ec0bdd694bb36cd8fad289ec
Status: Downloaded newer image for busybox:1.37
3b2e6352e756a174796d45e53fb1b9914408a9d8c22224b7d175330abf217d74

$ curl --retry 5 --retry-connrefused --retry-delay 1 -fsS http://127.0.0.1:19080
VisionGuard-23L-0727

$ docker logs visionguard-warmup
warmup-started

$ docker exec visionguard-warmup sh -c id; cat /www/index.html; echo persisted > /www/proof.txt
uid=0(root) gid=0(root) groups=0(root),10(wheel)
VisionGuard-23L-0727

$ docker ps --filter name=visionguard-warmup
CONTAINER ID   IMAGE          COMMAND                  CREATED                  STATUS
PORTS                     NAMES
```

Figure 33. Actual Docker warm-up transcript, first portion: image, container, port, logs, and shell.

Hayyan Haider · 23L-0727_A2_Ops · 7 November 2026

Docker warm-up - execution transcript (2/2)

Actual saved execution output: docker_warmup.log

```
3b2e6352e756   busybox:1.37   "sh -c 'echo VisionG…"   Less than a second ago   Up 1 second
127.0.0.1:19080->8080/tcp   visionguard-warmup

$ docker stop visionguard-warmup
visionguard-warmup

$ docker rm visionguard-warmup
visionguard-warmup

$ docker run --rm -v visionguard_warmup_data:/www busybox:1.37 cat /www/proof.txt
persisted

$ docker image rm busybox:1.37
Untagged: busybox:1.37
Deleted: sha256:a7dbc319b1438115a207e11930dc4bb3e643dcb3ec0bdd694bb36cd8fad289ec

$ docker volume rm visionguard_warmup_data
visionguard_warmup_data
```

Figure 34. Actual Docker warm-up transcript, continuation: stop, removal, volume recovery and cleanup.

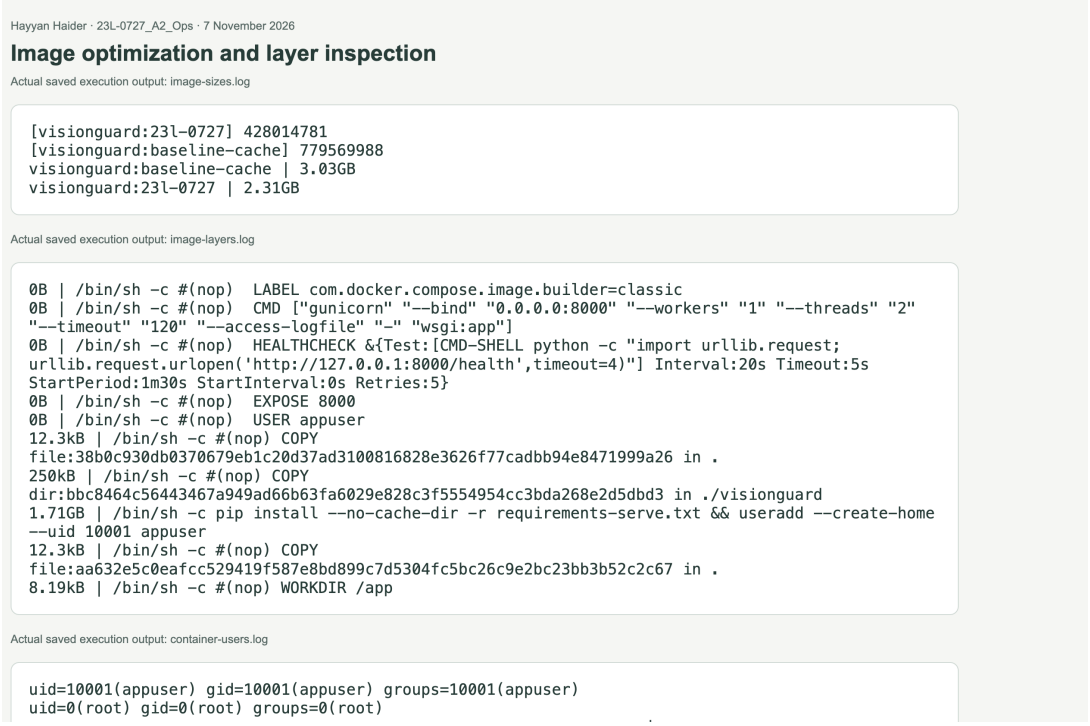
6.2 - Dockerfile, image layers, and optimization

The image starts from Python 3.11 slim Bookworm. Dependency installation precedes application COPY, allowing cached dependency layers when code changes. TensorFlow is installed without CUDA extras; no GPU runtime is required. The application runs as UID 10001, exposes the web port, and has a health check. Gunicorn uses one worker with two threads to avoid loading multiple model copies into memory.

.dockerignore excludes credentials, virtual environments, Git/DVC caches, raw/processed datasets, outputs, notebooks, submission files, and local model binaries. The champion is supplied as a read-only bind mount, keeping large models and secrets out of image layers. PostgreSQL credentials are environment values, not baked into the Dockerfile.

Measurement	Baseline	Optimized
Docker CLI displayed image size	3.03 GB	2.31 GB
Change	pip cache retained	--no-cache-dir; same serving packages

Under the same local Docker image accounting, removing the pip download cache saves approximately 0.72 GB (24%). TensorFlow and its dependencies still dominate the image; further slimming would need dependency/runtime evaluation. Docker inspect returned a different size accounting (779,569,988 vs 428,014,781 bytes), so these figures are not mixed with the CLI comparison.



Hayyan Haider · 23L-0727_A2_Ops · 7 November 2026

Image optimization and layer inspection

Actual saved execution output: image-sizes.log

```
[visionguard:23l-0727] 428014781
[visionguard:baseline-cache] 779569988
visionguard:baseline-cache | 3.03GB
visionguard:23l-0727 | 2.31GB
```

Actual saved execution output: image-layers.log

```
0B | /bin/sh -c #(nop)  LABEL com.docker.compose.image.builder=classic
0B | /bin/sh -c #(nop)  CMD ["gunicorn" "--bind" "0.0.0.0:8000" "--workers" "1" "--threads" "2"
"--timeout" "120" "--access-logfile" "-" "wsgi:app"]
0B | /bin/sh -c #(nop)  HEALTHCHECK &{Test:[CMD-SHELL python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health',timeout=4)"] Interval:20s Timeout:5s
StartPeriod:1m30s StartInterval:0s Retries:5}
0B | /bin/sh -c #(nop)  EXPOSE 8000
0B | /bin/sh -c #(nop)  USER appuser
12.3kB | /bin/sh -c #(nop) COPY
file:38b0c930db0370679eb1c20d37ad3100816828e3626f77cadbb94e8471999a26 in .
250kB | /bin/sh -c #(nop) COPY
dir:bbc8464c56443467a949ad66b63fa6029e828c3f5554954cc3bda268e2d5dbd3 in ./visionguard
1.71GB | /bin/sh -c pip install --no-cache-dir -r requirements-serve.txt && useradd --create-home
--uid 10001 appuser
12.3kB | /bin/sh -c #(nop) COPY
file:aa632e5c0eafcc529419f587e8bd899c7d5304fc5bc26c9e2bc23bb3b52c2c67 in .
8.19kB | /bin/sh -c #(nop) WORKDIR /app
```

Actual saved execution output: container-users.log

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
uid=0(root) gid=0(root) groups=0(root)
```

Figure 35. Actual image-size and layer output; dependency layer dominates.

6.3 - Multi-service Compose and operations

compose.yaml defines web and db services. The web app connects to hostname db rather than localhost. PostgreSQL pg_isready gates web startup through a health dependency; a named volume retains its data. The database has no host-published port. Web binds to localhost, and the model mount is read-only. A fresh .env plus model files is sufficient for docker compose up --build -d. The demonstration used WEB_PORT=8002 because the standalone Flask process already occupied 8000.



Figure 36. Actual service status, web UID, and network/port evidence.

6.3 - Deliberate failure and recovery

Stop/start/restart, service logs, and shell execution in both services were exercised. A deliberate MODEL_PATH=/models/missing.keras override caused a real startup failure with FileNotFoundError. Restoring the correct mount path and recreating web returned health status ok. Failure evidence is retained separately from the corrected healthy state.



Figure 37. Actual deliberate missing-model failure, followed by repair and successful health response.

6.4 - GitHub Actions: red, green, and project checks

The separate ci-toy repository tests Python 3.11 and 3.12. Its first commit implements addition incorrectly as subtraction, causing the unit test to fail. A second commit fixes addition; both matrix jobs pass. These are real GitHub-hosted workflow executions, not simulated status badges.

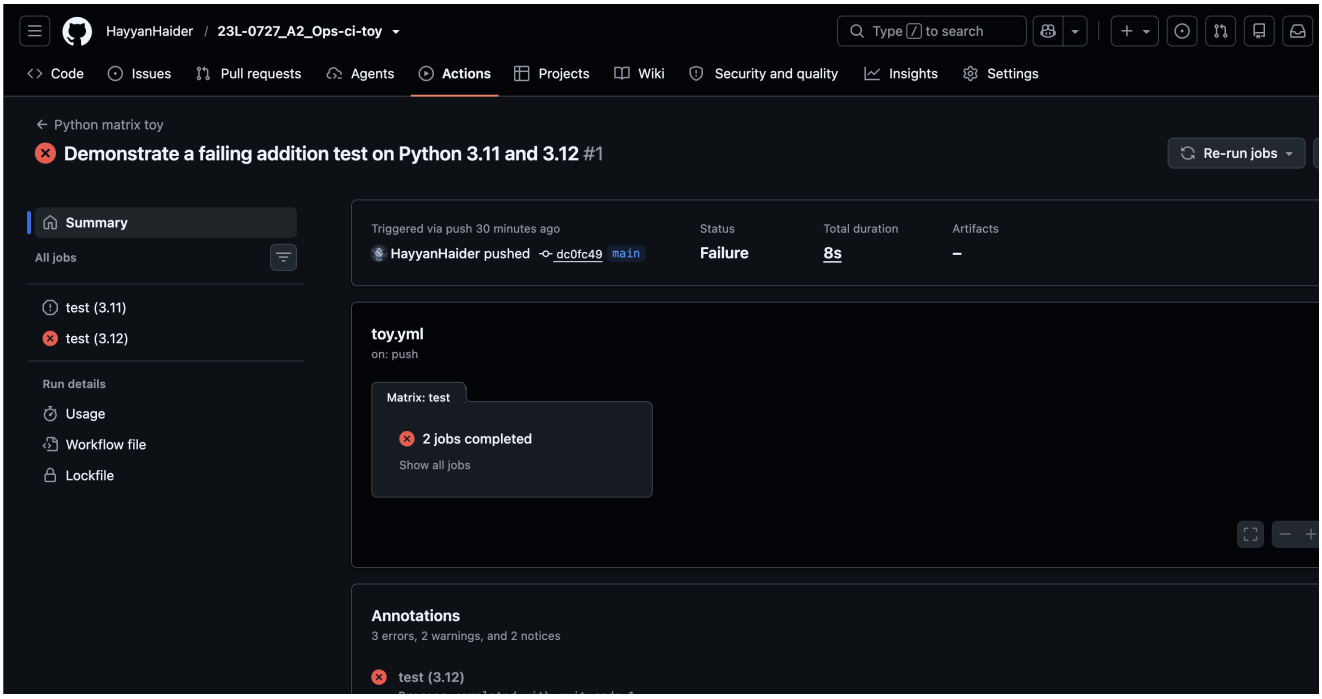


Figure 38. Actual failing toy workflow; the accompanying log records the deliberate assertion failure.

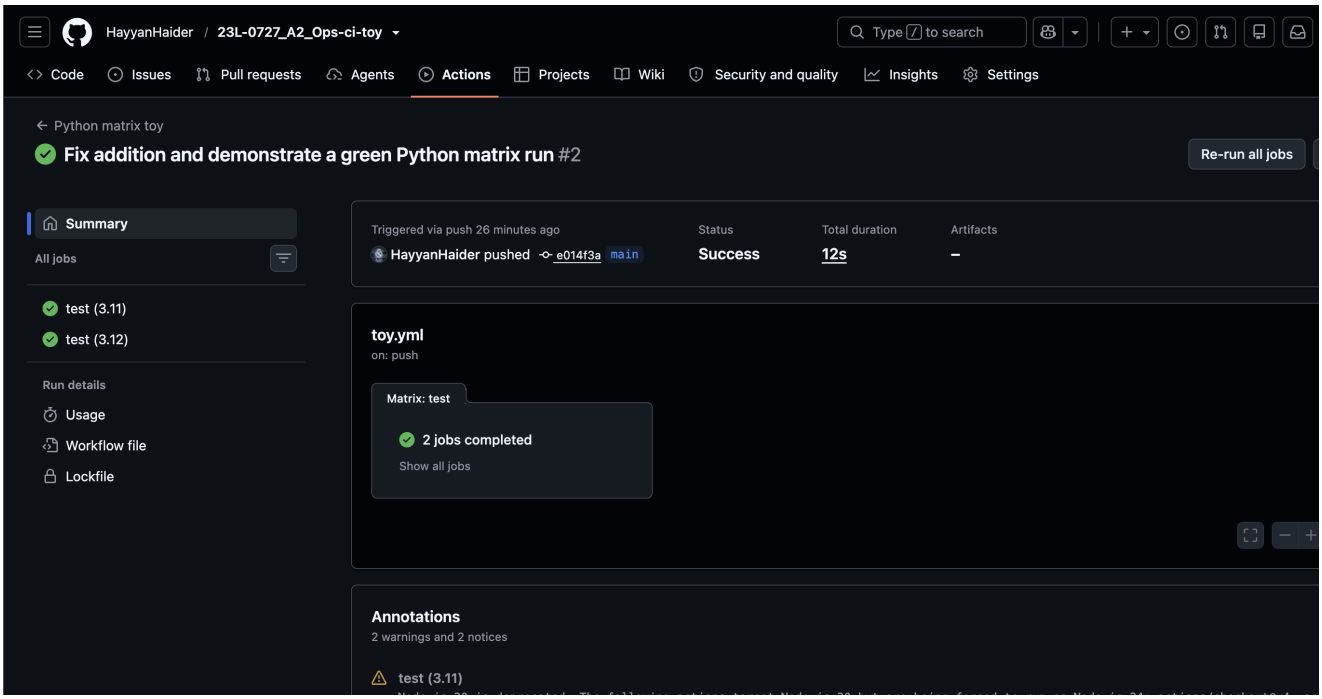


Figure 39. Actual passing toy workflow after the fix; Python version matrix succeeds.

6.4 - Main project CI

Main project CI installs serving/test dependencies, performs critical-module Ruff checks, runs meaningful Flask validation/inference/history tests, and builds the Docker image. Ten tests pass locally; the main GitHub-hosted workflow also succeeds. The README links its live status badge. The test predictor is deterministic, while separate HTTP checks exercise the real saved model.

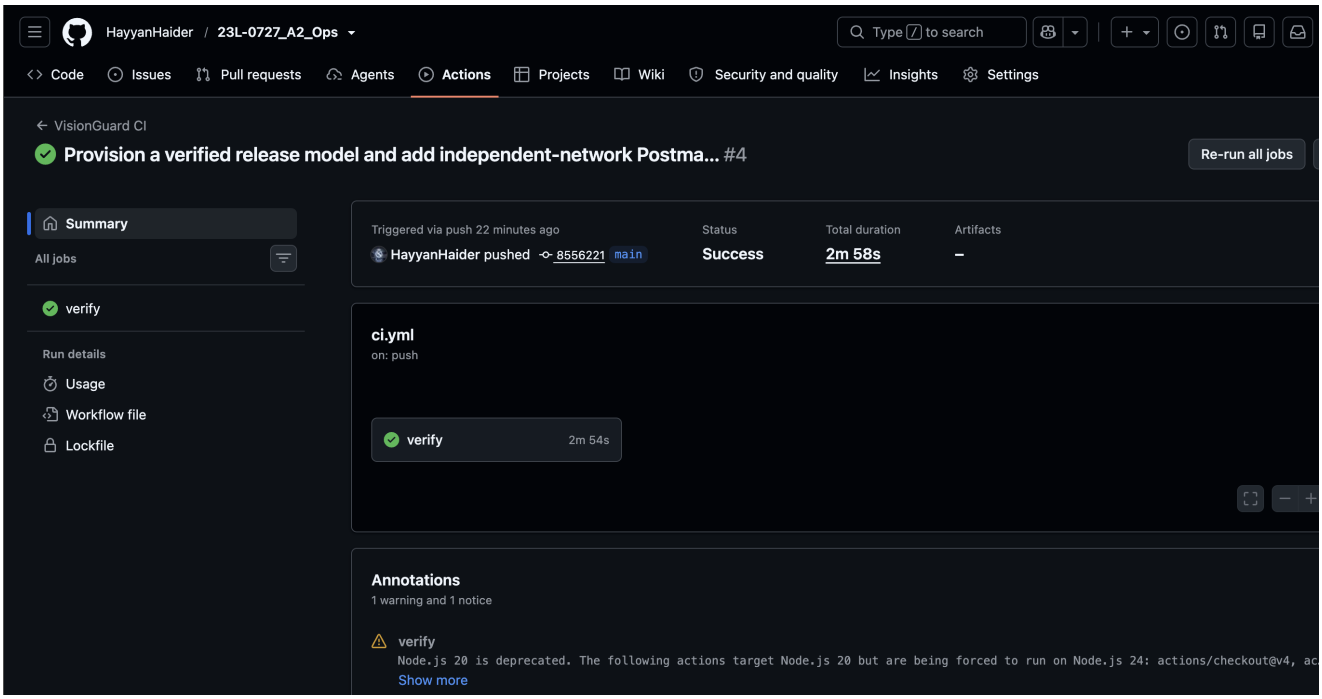


Figure 40. Actual successful project CI, including the Docker image build.

Task 7 - Cloud model API

7.1 - Deployment steps and endpoint

GitHub Codespaces was used under the user-approved free allowance: HayyanHaider Pro, two cores, Southeast Asia. Billing showed minimal included usage; no paid machine was selected. The existing repository was opened in a Codespace, its own source folder trusted, and requirements-serve.txt installed. scripts.provision_model downloaded the original champion from the project release and verified SHA-256. scripts.start_cloud started Gunicorn on 0.0.0.0:8001 with API_ONLY=true, a workspace SQLite history database, and the API key supplied through a Codespaces secret.

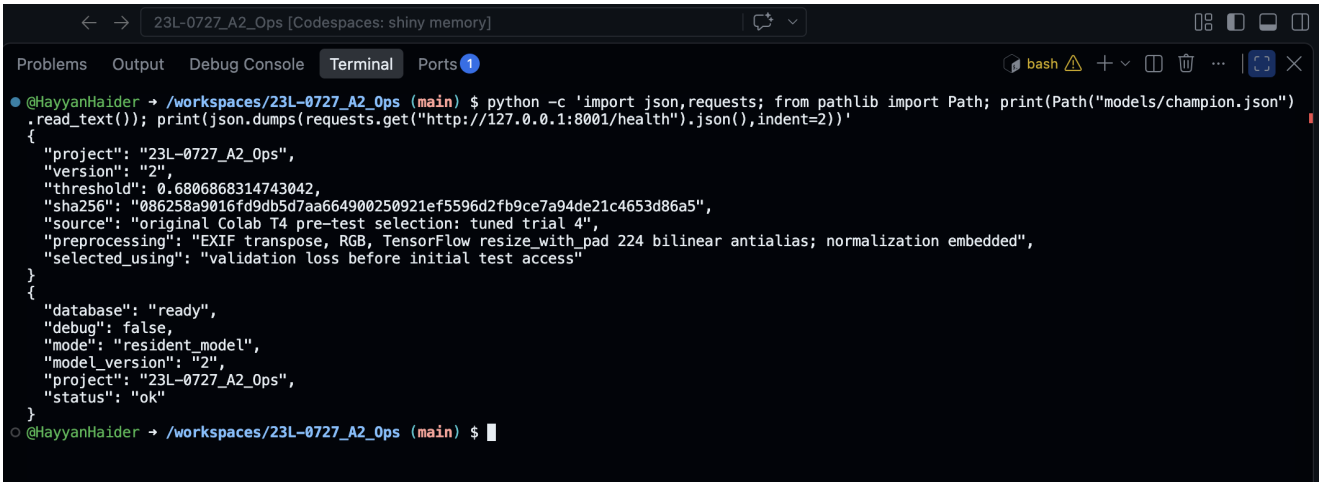


Figure 41. Actual cloud terminal: provisioned model, matching hash, and successful local API health.

Port 8001 is forwarded with HTTP to the backend and HTTPS at the public boundary. Public prediction requests require X-API-Key; the unauthenticated health route reveals only readiness information. The key is not included in report, screenshots, repository, collection, or ZIP. All other routes are protected in API-only mode.

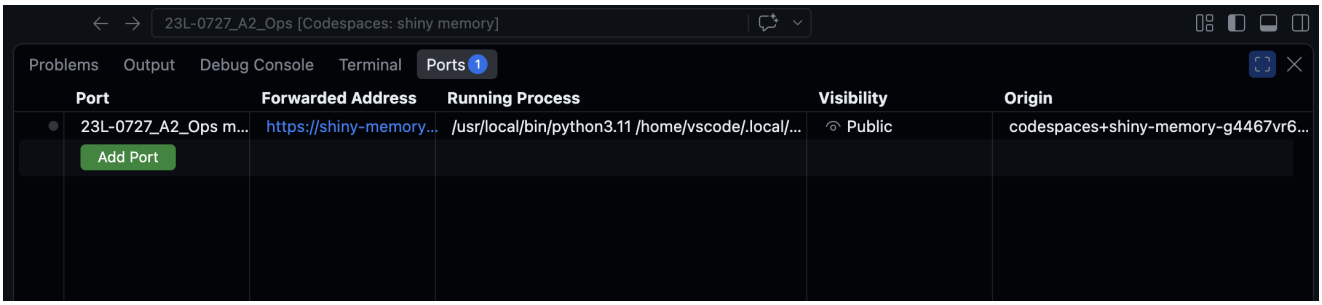
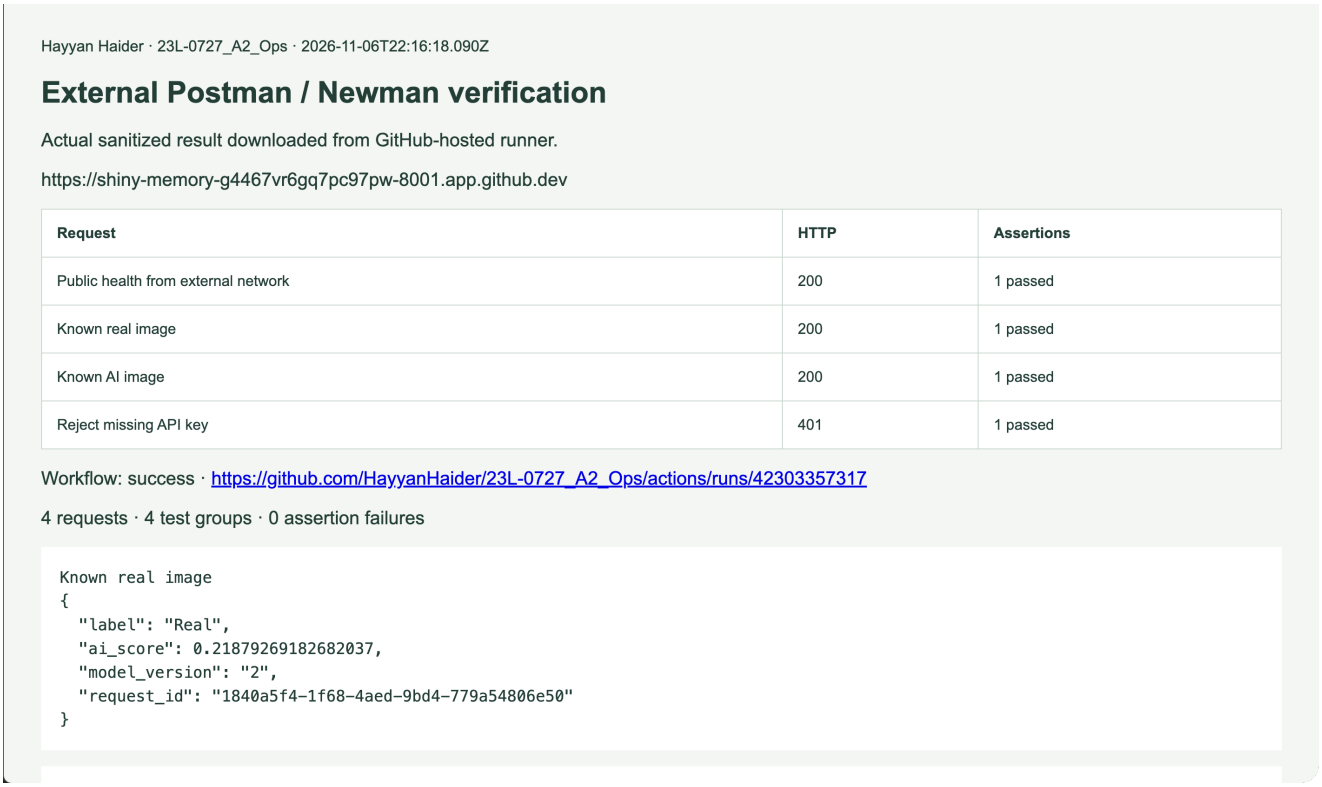


Figure 42. Actual Codespaces port configuration and public forwarded address.

API base URL: https://shiny-memory-g4467vr6gq7pc97pw-8001.app.github.dev
Prediction: POST /api/predict, multipart field image, X-API-Key header.
Health: GET /health.

7.2 - Postman collection and independent network test

The supplied Postman collection runs through Newman, Postman's official command-line runner. It checks public health, known-real and known-AI PNG requests, response schema/scores/model version/request ID, and rejection without a key. API_URL and INFERENCE_API_KEY are injected at runtime; exported evidence omits request authentication headers.



Hayyan Haider · 23L-0727_A2_Ops · 2026-11-06T22:16:18.090Z

External Postman / Newman verification

Actual sanitized result downloaded from GitHub-hosted runner.

https://shiny-memory-g4467vr6gq7pc97pw-8001.app.github.dev

Request	HTTP	Assertions
Public health from external network	200	1 passed
Known real image	200	1 passed
Known AI image	200	1 passed
Reject missing API key	401	1 passed

Workflow: success · https://github.com/HayyanHaider/23L-0727_A2_Ops/actions/runs/42303357317

4 requests · 4 test groups · 0 assertion failures

```
Known real image
{
  "label": "Real",
  "ai_score": 0.21879269182682037,
  "model_version": "2",
  "request_id": "1840a5f4-1f68-4aed-9bd4-779a54806e50"
}
```

Figure 43. Actual Postman/Newman verification summary from a GitHub-hosted runner, independent of the local network.

Request	HTTP status	Observed outcome
known_real.png	200	Real
known_ai.png	200	Real
wrong.exe	400	Choose a JPG, PNG, or WebP image.
corrupt.png	400	This file cannot be decoded as a valid image.
oversize.png	413	The upload is too large. Choose a file smaller than 5 MB.

7.2 - Independent-network workflow

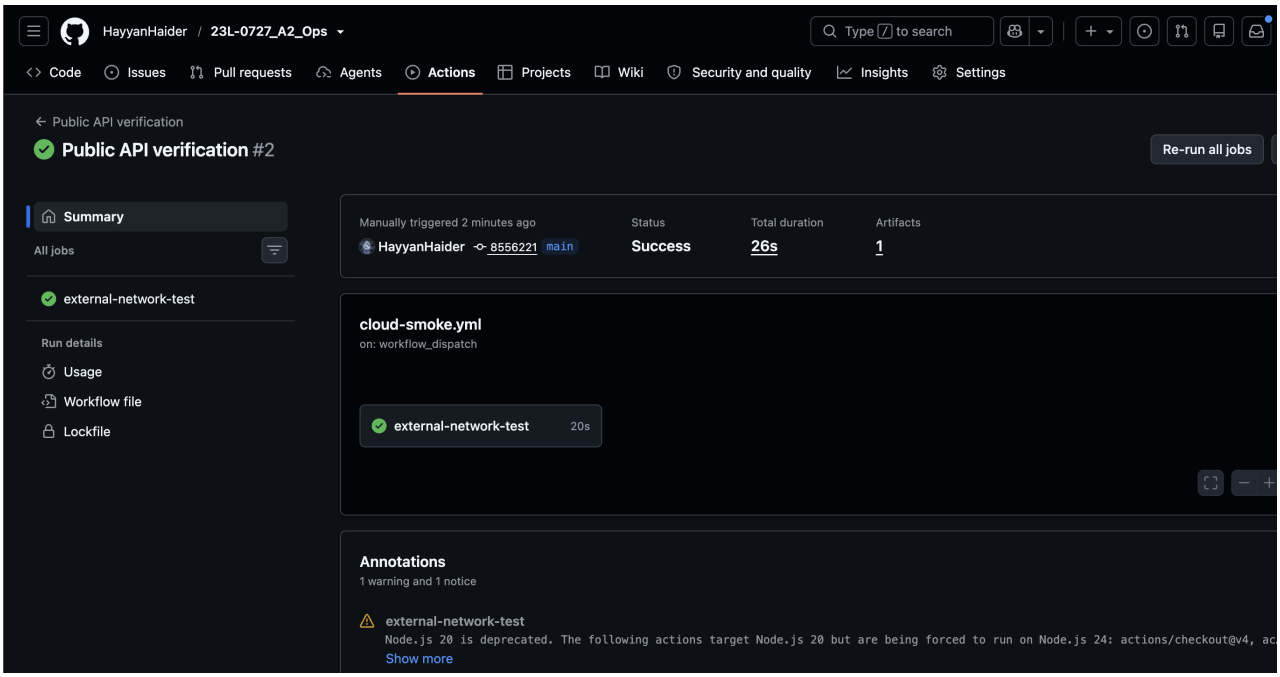


Figure 44. Successful external-network workflow in the project repository.

A separate GitHub Actions workflow executes the collection against the public URL. Its successful run and sanitized JSON artifact demonstrate access from a second network. Local HTTP verification separately sends two valid images and the three invalid cases to the deployed model, without recalculating held-out evaluation metrics.

Availability limit. Codespaces is an on-demand development host, not an always-on production service. Idle timeout can stop the API; restart the Codespace, run python -m scripts.start_cloud if needed, and set port 8001 public again before grading. Visibility may revert to private after restart. The README documents these steps. Keep an API key private and share a grading credential separately if required.

Task 8 - Local Flask to cloud API integration

The local Flask app sets INFERENCE_API_URL to the public Codespaces URL and uses the matching key from its ignored environment. In remote mode it does not load the local predictor. It validates the upload, sends the in-memory multipart file to /api/predict with a 10-second connection/90-second response timeout, checks required response fields, persists the result, and renders it. Upstream errors produce a friendly 503; no silent local-model fallback masks a failed cloud call.

Pretty print ☑


```
{
  "database": "ready",
  "debug": false,
  "mode": "remote_api",
  "model_version": "remote",
  "project": "23L-0727_A2_Ops",
  "status": "ok"
}
```


Figure 45. Actual local health endpoint: remote_api mode and ready history database.

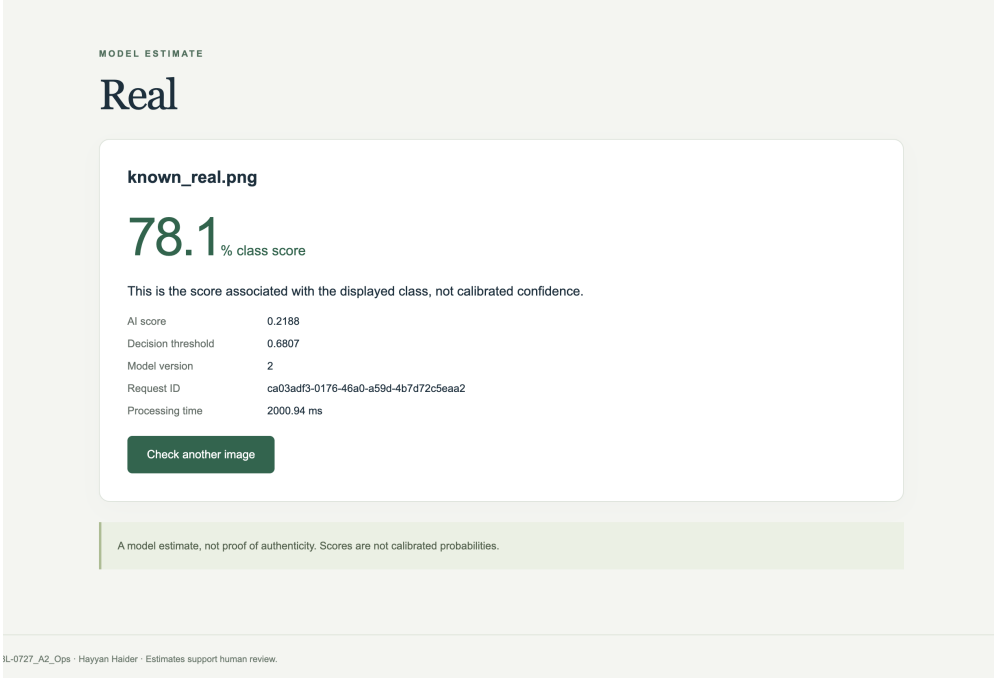


Figure 46. Actual local HTML result obtained through the deployed cloud API: known-real input.

8 - Second input through the cloud API

The AI-labelled training sample reaches the same remote champion. It is classified as Real: an AI score around 0.5264 falls below the validation-selected threshold. The outcome is retained as a model error; a successful HTTP request is not proof of a correct class decision.

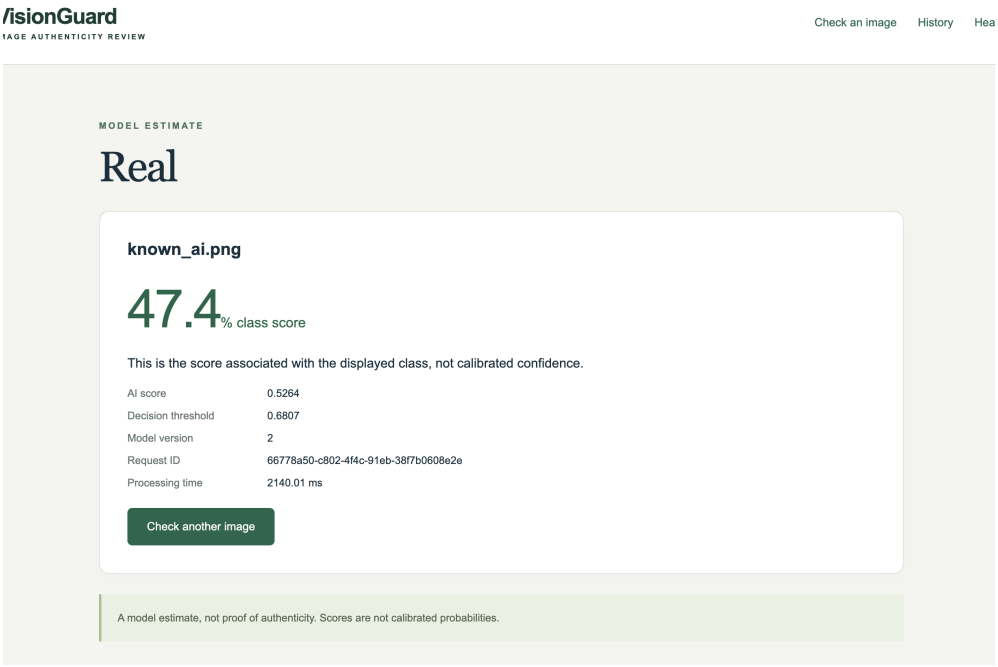


Figure 47. Actual local HTML result obtained through the deployed cloud API: known-AI input.

8 - Request records and integration interpretation

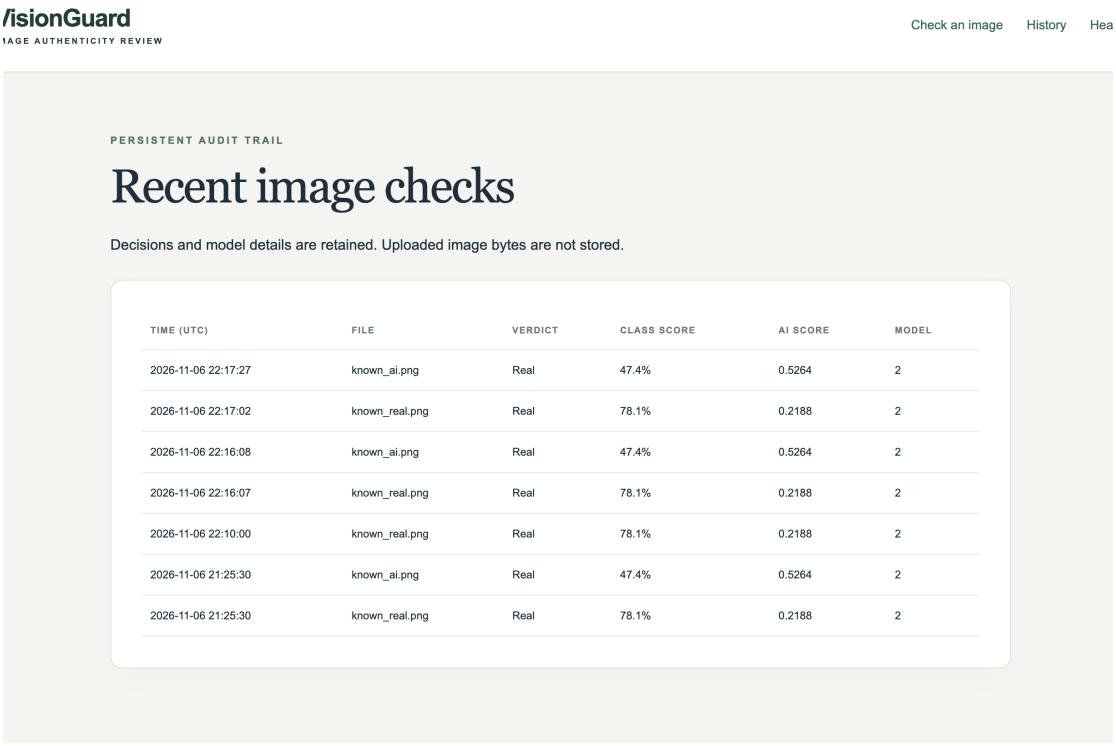


Figure 48. Actual local history containing both cloud-backed request results.

The known-AI demonstration input is classified as Real (AI score about 0.5264, below the 0.6807 threshold). This is a model error, retained in the evidence. Transport success does not imply classification correctness.

Both demonstration inputs come from the training split and are supplied only to show request/response behavior. Their results are not a new accuracy estimate. The model label, score, threshold, version and hash identify the same original champion used in Task 2. The local history receives its own request UUID and UTC timestamp, so local records can be distinguished from the remote API's request records.

Local and cloud validation tests confirm successful predictions and expected 400/413 errors. The independent-network Postman run verifies authentication rejection as well. PostgreSQL persistence, genuine model execution, cloud transport, and UI rendering are exercised separately, so a mocked unit test is not the only evidence for the final system.

Submission map, reproduction, and limitations

Deliverable	Contents/use
23L-0727_VisionGuard.pdf	Complete report with Tasks 1-8, figures, measured results and disclosure.
23L-0727_A2_Submission.zip	Source, model + metadata, executed notebook, selected training outputs, manifests, real verification logs, clean screenshots, Postman collection and report.
GitHub main + dvc-toy	Source/CI and separate two-version DVC lab history.
DagsHub project	Mirrored source, 20 MLflow runs, model registry and artifacts.

For a fresh local serving run: unpack the ZIP; copy .env.example to .env; set fresh POSTGRES_PASSWORD and INFERENCE_API_KEY; leave INFERENCE_API_URL empty for resident inference; ensure models/champion.keras and champion.json exist; run docker compose up --build -d. Open http://localhost:8000. Use docker compose logs and /health for diagnostics. Do not use down -v when preserving history.

For cloud integration: resume the named Codespace and public port, configure its API key privately, set local INFERENCE_API_URL and the same key, then recreate web. Reproduce training from the original dataset using the saved split lists and original environment; do not retune against the exposed test set. DVC recovery is demonstrated against permitted local-folder storage, not claimed as uploaded remote dataset bytes on DagsHub.

Issue	Resolution / practical limit
macOS TensorFlow + old PyArrow conflict	Upgraded local PyArrow to 26.0.1; recorded actual lock. Serving has no MLflow/PyArrow dependency. MLflow 2.22.5 declares an older Arrow bound, so this is a local compatibility workaround.
ARM CPU TensorFlow package	Use tensorflow==2.21.0 without CUDA extras; actual ARM Docker and Linux Codespace inference verified.
Image accounting differs by command	Compare the same Docker CLI display before/after; retain inspect output separately.
Forwarding initially returned sign-in HTML	Changing the backend protocol reset visibility. Verified HTTP first, restored Public, then reran the external collection successfully.
On-demand cloud host	API stops when Codespace sleeps. Resume before grading; no always-on uptime claim.
Model limitations	Small single-source test, uncertain labels and possible source/near-duplicate leakage. Confidence is uncalibrated; human review remains necessary.

The optional Grad-CAM bonus was not attempted. All eight required tasks are covered; historical experiment imports and later validation-only diagnostics are explicitly distinguished from the original pre-test model selection.

Immutable CI evidence: toy red run 42298330612, toy green run 42298918723, project green run 42299402072, and external API green run 42303357317. Full workflow URLs, commit hashes, job conclusions and timestamps are retained in ci-runs.json and cloud-external/workflow.json.

Additional references

MLflow TensorFlow autologging: https://mlflow.org/docs/latest/python_api/mlflow.tensorflow.html

MLflow registry aliases: https://mlflow.org/docs/latest/ml/model-registry/workflow/

DagsHub experiment tracking: https://dagshub.com/docs/integration_guide/mlflow_tracking/

Docker Compose startup ordering: https://docs.docker.com/compose/how-tos/startup-order/

Postman Newman: https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/

GitHub Codespaces port forwarding: https://docs.github.com/en/codespaces/developing-in-a-codespace/forwarding-ports-in-your-codespace

Included GitHub product usage: https://docs.github.com/en/billing/reference/product-usage-included